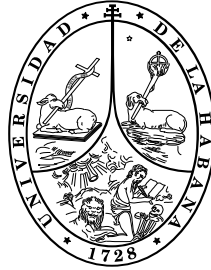


Universidad de La Habana
Facultad de Matemática y Computación



LDMAG: Lenguaje para el diseño de microaplicaciones de gestión

Autor: Ricardo Antonio Cápiro Colomar

**Tutores:
MSc. Fernando Rodríguez Flores**

Trabajo de Diploma
presentado en opción al título de
Licenciado en Ciencia de la Computación



La Habana,
junio 2025

"La mejor forma de predecir el futuro es inventarlo."

Alan Kay

Agradecimientos

Opinión del Tutor

La automatización de pequeños procesos cotidianos tiene importancia económica y psicológica. Por un lado, reduce costos operativos al ahorrar tiempo, optimizar recursos y minimizar errores humanos, además de aumentar la productividad individual y colectiva, permitiendo escalar tareas repetitivas (como usar apps para gestionar finanzas o herramientas de IA para organizar correos). Por otro lado, desde una perspectiva psicológica, libera la mente de cargas cognitivas triviales, reduce el estrés vinculado a decisiones rutinarias (ejemplo: recordatorios automáticos) y fortalece la sensación de control y eficiencia, mejorando el bienestar emocional y facilitando el enfoque en actividades creativas o estratégicas. Así, la automatización no solo potencia la eficiencia económica, sino que también contribuye a un equilibrio mental más saludable y productivo.

En nuestra facultad (y probablemente en gran parte de nuestras vidas) existen muchos pequeños procesos cotidianos que sería muy cómodo optimizar. El trabajo que se presenta en este documento es un primer paso en la dirección de automatizar muchos de estos procesos. De hecho, este trabajo consiste en automatizar la automatización, así que probablemente herede todas las ventajas económicas y psicológicas planteadas en el párrafo anterior.

Para llevar a cabo este trabajo, el diplomante tuvo que usar tecnologías y modos de actuación, no solo desconocidos para él, sino que ni siquiera tenía idea de que pudieran existir. Funciones genéricas, macros, polimorfismo en múltiples argumentos son solo algunas de las herramientas con las que ahora cuenta Ricardo, y que tuvo que adquirir mientras resolvía un problema complejo.

Estoy muy contento con los resultados obtenidos y con el trabajo realizado. Durante este tiempo Ricardo ha demostrado que ya tiene todos los requisitos y condiciones para desempeñarse como un excelente científico de la computación.

MSc. Fernando Raul Rodriguez Flores
Facultad de Matemática y Computación
Universidad de la Habana
Junio, 2025

Resumen

La gestión de la información en entornos educativos requiere de sistemas que centralicen y estructuren los datos, pero las soluciones actuales (hojas de cálculo, bases de datos, apps de consola, web o bots) dependen del escenario específico en que se presenten. El objetivo de esta tesis es automatizar la creación de microaplicaciones de gestión mediante un lenguaje de dominio específico que, a partir de una descripción sencilla, genere código para múltiples formatos (bases de datos SQL, aplicaciones de consola y web). Implementado en Common Lisp con macros y CLOS, este DSL modular y extensible simplifica el proceso de desarrollo, permitiendo obtener aplicaciones funcionales.

Palabras claves: Lenguaje de dominio específico; Microaplicaciones de gestión; Generación automática de código; Common Lisp; Macros; CLOS

Abstract

Information management in educational settings requires systems that centralize and structure data, yet current solutions (spreadsheets, databases, console apps, web apps, or bots) depend heavily on the specific scenario. The objective of this thesis is to automate the creation of management micro-applications through a domain-specific language that, from a simple description, generates code for multiple targets (SQL databases, console applications, and web applications). Implemented in Common Lisp using macros and CLOS, this modular and extensible DSL streamlines the development process, enabling the rapid production of fully functional applications.

Keywords: Domain-Specific Language; Management Micro-Applications; Automated Code Generation; Common Lisp; Macros; CLOS

Índice general

Introducción	1
1. Preliminares teóricos	6
1.1. Definiciones básicas	6
1.2. Gestión de procesos académicos	7
1.3. Lenguajes de Dominio Específico	8
1.4. Árboles de Sintaxis Abstracta (AST)	9
1.5. Lenguaje de Programación Lisp	10
1.5.1. Sistema de Objetos de Common Lisp (CLOS)	11
1.5.2. Clases y Notación Prefija	12
1.5.3. Funciones en Lisp	12
1.5.4. Funciones Genéricas y Métodos	13
1.5.5. Macros	14
2. Aspectos de las microaplicaciones de gestión	16
2.1. Ejemplos de microaplicaciones de gestión	16
2.2. Elementos de las microaplicaciones de gestión	22
2.2.1. Almacenamiento de la información	22
2.2.2. Definición de las consultas	23
2.2.3. Representación visual de una microaplicación	24
2.3. Descripción de una microaplicación de gestión	26
3. Aspectos del lenguaje propuesto	30
3.1. Abstracción y generación automática de clases en el lenguaje	30
3.2. Elementos del lenguaje	32
3.2.1. Definición de texto	33
3.2.2. Definición de título	33
3.2.3. Definición de entrada de datos	33
3.2.4. Definición de elemento de listas	33

3.2.5. Definición de una página	34
3.2.6. Definición de entidades	34
3.2.7. Definición de una instancia de una entidad	35
3.2.8. Declaración de acceso a un atributo de una entidad	35
3.2.9. Declaración de operadores de comparación	36
3.2.10. Declaración de operadores lógicos	37
3.2.11. Definición de consultas	38
3.3. Descripción de una microaplicación de gestión en el lenguaje propuesto	39
4. Generación de código	42
4.1. Clases abstractas y métodos para la generación de código	43
4.2. Formatos para gestionar las microaplicaciones de gestión	45
4.2.1. Exportación a consola de Python	46
4.2.2. Exportación a aplicaciones web con Django	48
4.2.3. Exportación a sistemas de gestión de bases de datos como MySQL	51
5. Detalles de implementación	53
5.1. Estructura general de los nodos del lenguaje	53
5.1.1. Nodos con constructores de argumentos posicionales	53
5.1.2. Nodos con constructores de argumentos &rest	54
5.2. Automatización mediante macros	54
5.2.1. Macro defclass*	55
5.2.2. Macro def-entidad	55
5.2.3. Macro def-consulta	57
Conclusiones	58
Recomendaciones	59
Referencias Bibliográficas	60

Introducción

La gestión de la información es el conjunto de procesos que sirve para designar actividades orientadas a la generación, coordinación, almacenamiento, conservación, búsqueda y recuperación de datos e información dentro de una organización o contexto determinado [6]. En un entorno cada vez más digitalizado y globalizado, la información se ha convertido en un recurso estratégico fundamental para la toma de decisiones, la innovación y la competitividad [24].

En el campo educativo, la gestión de la información adquiere una dimensión estratégica, ya que abarca tanto los procesos administrativos como los pedagógicos [31]. Incluye la administración de datos relacionados con la matrícula, horarios, calificaciones, control de asistencia, gestión de incidencias y documentación institucional [8].

Ante el volumen de datos que se manejan en instituciones académicas y empresariales, surge la necesidad de contar con sistemas o aplicaciones especializadas para la gestión de la información. Estos sistemas permiten organizar y centralizar la información de manera estructurada. La utilización de aplicaciones de gestión de la información no solo responde a la demanda de optimización de los procesos internos, sino que también contribuye a la mejora continua de la calidad institucional, al proporcionar datos confiables para la toma de decisiones y la identificación de áreas de mejora [13].

La incorporación de sistemas de gestión escolar y plataformas digitales permite automatizar tareas rutinarias, reducir errores y liberar recursos para actividades pedagógicas de mayor valor añadido [3].

En la Facultad de Matemática y Computación de la Universidad de La Habana, se han realizado varias investigaciones de tesis de pregrado en las que se han abordado problemas concretos de gestión relacionados con la gestión del horario docente [29, 28], la asignación de profesores en las asignaturas del semestre [41], la planificación de las defensas de tesis [41,

15], la gestión de cursos docentes basados en créditos y entretenimiento [27, 37], y la planificación de exámenes mundiales [21].

Existen varias alternativas para implementar un sistema o aplicación de gestión de la información, cada una con niveles distintos de complejidad, ventajas y desventajas, por lo que la elección depende del problema en cuestión. Se pueden señalar como alternativas:

- **Hojas de cálculo (Excel, Calc):** Son herramientas para la gestión básica de datos, permitiendo organizar, filtrar, analizar y visualizar información de manera sencilla. Resultan útiles para proyectos pequeños o cuando se requiere una solución rápida y de bajo costo, por lo que son muy limitadas para flujos de trabajo específicos. Ejemplos de tareas que se pueden hacer con Excel son organizar calendarios o agendas, realizar informes contables y la planificación de eventos.
- **Sistemas de bases de datos (SQLite, MySQL, PostgreSQL):** Adecuados para gestionar grandes volúmenes de información de manera estructurada, con capacidades avanzadas de consulta, seguridad y respaldo [11]. El inconveniente es que no ofrecen interfaces listas para el usuario final.
- **Aplicaciones de consola:** Ideales para entornos donde la interfaz gráfica no es prioritaria o se requiere automatizar tareas específicas mediante *scripts* o programas ejecutados desde la línea de comandos, por lo que los usuarios se tornan limitados a, por ejemplo, especialistas de la computación que requieren, en ocasiones, de aplicaciones funcionales y visualmente sencillas.
- **Aplicaciones web personalizadas:** Utilizando frameworks como Django [10], Flask [26] o Laravel [18], es posible desarrollar aplicaciones a medida que se adapten a las necesidades específicas de gestión de información, con acceso multiusuario, interfaces personalizadas y almacenamiento centralizado de datos. El inconveniente es que el tiempo de desarrollo es mayor y se pierde tiempo que pudiera ser aprovechado en otras actividades.
- **Bots de mensajería (Telegram):** Permiten la interacción y gestión de información a través de plataformas de mensajería, facilitando la automatización de consultas, notificaciones y registro de datos en tiempo real [22]. En este caso, el tiempo de desarrollo es menor, pero dependen de un servidor que tiene que estar ejecutándose para poder

utilizarlo. Ejemplos de aplicaciones hechas en bots de Telegram se encuentra un bot interactivo como herramienta de apoyo en la asignatura de Inteligencia Artificial para recibir guías de prácticas en la materia [36]. Otro ejemplo es un bot para permitir a los estudiantes de nuevo ingreso de una universidad española consultar información de forma rápida y sencilla sobre la universidad [30].

Todas estas herramientas facilitan el desarrollo de aplicaciones de gestión de la información. Sin embargo, su elección y uso dependen del escenario específico, y todas ellas, a pesar de sus diferencias, operan sobre elementos comunes en la gestión de datos. Esta observación sugiere la conveniencia de una herramienta unificada capaz de describir el problema y el formato de salida deseado, para luego generar automáticamente la aplicación de gestión. Precisamente, esta tesis busca automatizar el proceso de diseño y generación de aplicaciones de gestión de la información, especialmente en el contexto educativo, donde la implementación de soluciones personalizadas requiere conocimientos técnicos avanzados y un tiempo considerable de desarrollo.

El problema que se aborda es la falta de herramientas que permitan a los usuarios describir sus necesidades y obtener aplicaciones para gestionar procesos de manera automática en diversos formatos y lenguajes como los mencionados anteriormente. Por ello, la pregunta central de esta investigación es: ¿Puede un lenguaje de dominio específico facilitar la descripción y generación automática de aplicaciones de gestión en distintos formatos, optimizando el tiempo y los recursos requeridos?

Hipótesis de la investigación:

La utilización de un lenguaje de dominio específico (DSL) facilita la automatización y simplificación del desarrollo de aplicaciones de gestión en distintos formatos, permitiendo una mayor adaptabilidad a las necesidades del usuario final.

Objetivo general:

El objetivo general de este trabajo es diseñar e implementar un lenguaje de dominio específico para la descripción y generación automática de aplicaciones de gestión en diversos formatos, tales como sistemas de gestión de bases de datos SQL, aplicaciones de consola y aplicaciones web en Django.

Para el cumplimiento de este objetivo general, se han trazado los siguientes objetivos específicos:

- Analizar herramientas de gestión existentes, específicamente en el contexto educativo.
- Analizar el estado del arte sobre lenguajes de dominio específico y herramientas para la generación automática de aplicaciones de gestión.
- Identificar los elementos y requisitos fundamentales para la descripción de aplicaciones de gestión en diversos contextos como sistemas de gestión de bases de datos SQL, aplicaciones de consola o aplicaciones web en Django.
- Diseñar un lenguaje de dominio específico que permita describir aplicaciones de gestión de manera flexible y extensible.
- Implementar el lenguaje diseñado utilizando Common Lisp, aprovechando sus capacidades para la construcción de lenguajes y generación de código.
- Validar la funcionalidad y versatilidad del lenguaje mediante el desarrollo de aplicaciones de gestión en distintos escenarios y formatos de salida.

El lenguaje propuesto está implementado en Lisp, para aprovechar las ventajas de la notación prefija, el sistema de macros y las funcionalidades del sistema de objetos de Common Lisp (CLOS) [38, 39].

Para diseñar el lenguaje se identificaron los elementos necesarios para definir aplicaciones de gestión, y por cada uno de ellos se implementó una clase que representa a ese elemento. Aprovechando la notación prefija de Lisp, el lenguaje está formado por llamados anidados a los constructores de estas clases. Para exportar las aplicaciones descritas en otros lenguajes o formatos, se define una clase abstracta para identificar la generación de código de forma general y, aprovechando el polimorfismo en múltiples argumentos que permite CLOS [25], se define por cada lenguaje o formato a exportar cómo se realizaría la generación de código.

Estructura del documento:

Este documento se estructura de la siguiente manera:

En el capítulo 1 se sienta las bases teóricas y técnicas del trabajo. Se establecen definiciones clave sobre la gestión de procesos y los procesos académicos, introduciendo la noción de microaplicaciones de gestión. Se

argumenta la relevancia de los Lenguajes de Dominio Específico (DSL) como solución para automatizar la generación de estas aplicaciones, explicando su propósito y ejemplos. Finalmente, se abordan los Árboles de Sintaxis Abstracta (AST) como eje de la representación del lenguaje, y se justifica la selección de Common Lisp como herramienta de implementación, destacando sus características esenciales para el diseño del DSL.

En el capítulo 2 se profundiza en las microaplicaciones de gestión (MAG), presentando ejemplos concretos de contextos académicos y de otros ámbitos. A partir de estos, se identifican los componentes esenciales de toda microaplicación de gestión. La caracterización de estos elementos sienta las bases para el diseño del lenguaje propuesto.

El capítulo 3 detalla el lenguaje propuesto para las microaplicaciones de gestión. Se describen sus estructuras fundamentales para modelar datos, definir consultas y diseñar interfaces visuales, a partir de la creación de clases para cada una de ellas.

El capítulo 4 describe el proceso de generación de código, es decir, cómo es posible obtener aplicaciones de gestión en diferentes formatos a partir del lenguaje.

El capítulo 5 muestra algunos detalles de la implementación.

El documento concluye con la presentación de las conclusiones y recomendaciones para trabajos futuros. Finalmente, se incluyen las referencias bibliográficas consultadas durante la investigación.

Capítulo 1

Preliminares teóricos

En este capítulo se presentan los elementos fundamentales para el desarrollo de este trabajo. En la primera sección (1.1) se presenta un conjunto de definiciones que serán utilizadas a lo largo del documento. En la siguiente parte (1.2) se aborda la gestión de procesos académicos y se identifican algunos ejemplos que sirvieron de base para la creación del lenguaje propuesto. El resto de las secciones (1.3, 1.4 y 1.5) están dedicadas a los elementos necesarios para la creación de un lenguaje para diseñar aplicaciones de gestión: los Lenguajes de Dominio Específico (DSL), los Árboles de Sintaxis Abstracta (ASTs) y, por último, el lenguaje de programación Common Lisp.

1.1. Definiciones básicas

Con el objetivo de establecer un marco conceptual para el desarrollo de este trabajo, en esta sección se presentan algunas definiciones que permitirán describir mejor los procesos y conceptos abordados en los siguientes apartados.

Definición 1.1.1 *La gestión de procesos es el conjunto de actividades orientadas a planificar, organizar, ejecutar, controlar y mejorar los procesos que se desarrollan en una organización, con el objetivo de alcanzar de manera eficiente los resultados deseados [33].*

La gestión de procesos puede aplicarse en diversos ámbitos, como la industria, la salud, la administración pública o el sector financiero [33]. Por ejemplo, en una fábrica, la gestión de procesos abarca la planificación de la producción, el control de inventarios, la asignación de recursos y la supervisión de la calidad. En un hospital, implica la coordinación de citas

médicas, la gestión de historias clínicas, la asignación de personal y la administración de insumos médicos.

Definición 1.1.2 *Los **procesos académicos** son aquellos procesos que se desarrollan en el ámbito educativo y que están relacionados con la organización, ejecución y control de las actividades docentes, investigativas y de extensión universitarias.*

En una universidad, la gestión de procesos puede incluir la planificación de horarios, la asignación de profesores en las asignaturas del semestre, la inscripción de estudiantes en cursos o el seguimiento y análisis del rendimiento académico de los estudiantes.

Definición 1.1.3 *Una **microaplicación de gestión** es una aplicación informática diseñada para gestionar procesos en los cuales toda la información relevante puede representarse y almacenarse en una pequeña cantidad de unidades de almacenamiento.*

Por ejemplo, en sistemas de gestión basados en bases de datos relacionales como SQL, toda la información de una microaplicación de gestión puede almacenarse en pocas tablas. De manera similar, en aplicaciones desarrolladas en consola utilizando Python, los datos pueden guardarse en pocos archivos de texto (.txt).

A continuación, se profundiza en la gestión de procesos académicos, ilustrando cómo estos fundamentos teóricos se aplican en contextos concretos.

1.2. Gestión de procesos académicos

En la Facultad de Matemática y Computación de la Universidad de La Habana, la *gestión de procesos académicos* ha sido objeto de varias investigaciones en tesis de pregrado.

Dichos trabajos han abordado problemas concretos de gestión relacionados con la gestión del horario docente [29, 28], la asignación de profesores en las asignaturas del semestre [41], la planificación de las defensas de tesis [41, 15], la gestión de cursos docentes basados en créditos y entretenimiento [27, 37], y la planificación de exámenes mundiales [21].

Estas experiencias han evidenciado que, si bien cada solución puede diseñarse de forma independiente, existe una estructura común en muchos de estos procesos. A partir de esta observación surge la idea de crear un

lenguaje de dominio específico que permita describir estos *procesos académicos* de forma declarativa, de modo que, una vez descrito el proceso, se genere automáticamente una *microaplicación de gestión* correspondiente.

Para la construcción del lenguaje desarrollado en este trabajo se tuvo en cuenta la modelación de un conjunto de actividades académicas y de otros ámbitos, que sirvieron como base para abstraer los elementos esenciales de los procesos de gestión y definir los componentes del lenguaje propuesto.

Entre esos procesos se pueden mencionar la asignación de alumnos ayudantes, las defensas de tesis de diploma, la asignación de profesores en las asignaturas del semestre, la gestión de acontecimientos relevantes y la gestión de notas de libros.

La asignación de alumnos ayudantes consiste en determinar qué alumno ayudante debe apoyar a qué profesor a impartir una asignatura. La planificación de defensas de tesis implica organizar los tribunales, horarios y locales para las exposiciones. La asignación de docencia se refiere a distribuir profesores en las asignaturas y modalidades del semestre. La gestión de acontecimientos relevantes abarca registrar y consultar hechos significativos y fechas importantes. La gestión de notas de libros permite almacenar y clasificar anotaciones sobre libros para facilitar su consulta.

Todos estos procesos serán detallados en el capítulo siguiente, donde se identificarán los elementos comunes en las microaplicaciones de gestión y se establecerán las bases para el diseño del lenguaje de dominio específico.

1.3. Lenguajes de Dominio Específico

Los ingenieros de software se encargan de encontrar herramientas para representar soluciones a problemas de las ciencias computacionales. Una de las principales tendencias para la creación, modernización y mantenimiento de programas es el modelado de dominio específico (*DSM, por sus siglas en inglés*) [32]. Esta técnica propone crear abstracciones de los problemas para que los usuarios expresen fácilmente sus soluciones y ha sido utilizada en el desarrollo de lo que se conoce como Lenguajes de Dominio Específico (*DSL, por sus siglas en inglés*).

Un DSL es un lenguaje de programación destinado a aumentar la productividad de los usuarios al resolver un problema dentro de un contexto específico [9]. Los DSLs tienen la característica de ser típicamente más pequeños que los lenguajes de propósito general, pero al complementarse con otros ambientes de producción brindan un mejor nivel de abstracción sintáctico y semántico para la representación de un problema, permitiendo

de esta forma ocultar las complejidades que aparecen en el proceso. Precisamente por ese motivo, y porque las herramientas actuales facilitan el desarrollo de nuevos lenguajes, los DSLs se han convertido en una opción cada vez más adoptada en proyectos de software especializados [23].

A continuación se presentan algunos ejemplos de lenguajes de dominio específico ampliamente utilizados:

- **SQL (*Structured Query Language*)**: Un lenguaje diseñado específicamente para la gestión y consulta de bases de datos relacionales. Permite a los usuarios definir, manipular y consultar datos de manera declarativa [7].
- **HTML (*HyperText Markup Language*)**: Un lenguaje de marcado utilizado para estructurar y presentar contenido en la web. Aunque no es un lenguaje de programación en sentido estricto, es un DSL orientado al dominio de la presentación de documentos [40].
- **MATLAB**: Un lenguaje de programación específico para el cálculo numérico, análisis de datos y visualización, ampliamente utilizado en ingeniería y ciencias aplicadas [4].

En la próxima sección se presentan los Árboles de Sintaxis Abstracta (AST), que son importantes en la representación y procesamiento de programas.

1.4. Árboles de Sintaxis Abstracta (AST)

Los Árboles de Sintaxis Abstracta (AST) representan jerárquicamente, mediante un árbol, la estructura sintáctica abstracta o simplificada de un programa escrito en un lenguaje de programación. Estos árboles son fundamentales en el proceso de creación de lenguajes y en la fase de generación de código [14].

En los AST, cada nodo representa una instrucción del lenguaje. Estos nodos contienen la información necesaria para la generación de código de la instrucción que representa [2]. Por ejemplo, si el lenguaje posee la instrucción declaración de variable, durante la construcción del AST es necesario definir un nodo referente a esta instrucción mediante una clase.

El árbol es abstracto en el sentido de que no representa cada detalle que aparezca en la sintaxis verdadera, es decir, no contiene los paréntesis y otros símbolos que inevitablemente aparecen en el código. Los ASTs poseen

gran importancia en la construcción de los lenguajes debido a que pueden intervenir en varias fases del análisis: como producto del análisis sintáctico, como elemento intermedio en sucesivos análisis semánticos y como entrada para la generación de código [1].

En la figura 1.1 se muestra un ejemplo de un árbol de sintaxis abstracta que representa a la instrucción de asignación $d = a + b * c$. El AST que se muestra tiene como raíz el nodo que representa la operación ($=$); tanto la parte izquierda (d) como la parte derecha ($a + b * c$) son representadas como subárboles hijos del nodo raíz. De una forma similar ocurre con el nodo ($+$) y el nodo ($*$).

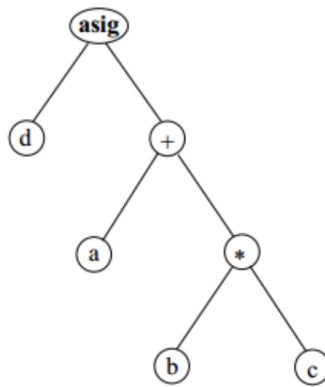


Figura 1.1: Árbol de Sintaxis Abstracta.

En el contexto de este trabajo, el objetivo principal es generar el código de microaplicaciones en diversos formatos de salida a partir del Árbol de Sintaxis Abstracta (AST). Tradicionalmente, el AST se obtiene como resultado del proceso de análisis sintáctico de la cadena de entrada. Sin embargo, en este caso particular, no es necesario realizar dicho análisis, ya que se aprovechan las capacidades que ofrece el lenguaje Common Lisp para el diseño de lenguajes de dominio específico.

En la siguiente sección se describen las características principales de Lisp y su relevancia para el desarrollo de lenguajes de dominio específico.

1.5. Lenguaje de Programación Lisp

Lisp es un lenguaje de programación creado originalmente en 1958 por John McCarthy y sus colaboradores en el Instituto Tecnológico de Massa-

chusetts. Su nombre se deriva de LISt Processing (Procesamiento de listas). Es un lenguaje de programación multiparadigma, que permite escribir programas de manera imperativa, funcional o declarativa [16, 34]. Debido a la variedad de dialectos Lisp existentes en 1981 (MacLisp, InterLisp, ZetaLisp) se creó el estándar Common Lisp en 1994 [12].

Entre las características más significativas de Common Lisp se encuentran la sintaxis con notación prefija en la que todas las expresiones se encuentran encerradas entre paréntesis. Common Lisp fue seleccionado para realizar la implementación del lenguaje que se presenta en este trabajo, debido a las ventajas que ofrece para el desarrollo del mismo. Entre ellas se encuentran la presencia de herencia múltiple, el uso de funciones genéricas que permiten tener polimorfismo en múltiples argumentos, el uso de macros y las facilidades de técnicas de creación e inicialización de objetos que permiten el control de ese proceso por parte del usuario [16, 34].

En los siguientes epígrafes se abordan las principales características de Lisp que fueron necesarias para la implementación del lenguaje.

1.5.1. Sistema de Objetos de Common Lisp (CLOS)

El Sistema de Objetos de Common Lisp (CLOS) es uno de los principales componentes de este lenguaje [25]; este permite el uso de las técnicas de la programación orientada a objetos en los programas. Define conceptos tales como objetos, clases y métodos en sus interacciones.

Common Lisp es un lenguaje multiparadigma, es decir, combina características de la programación funcional y orientada a objetos [35]. Se diferencia de varios lenguajes de alto nivel ampliamente utilizados, como Python, C# o Java, en aspectos como el hecho de que los métodos no pertenecen a una clase específica, permite el polimorfismo con múltiples argumentos y no implementa mecanismos de protección sobre las propiedades de las clases. Una de sus ventajas destacables es el soporte para herencia múltiple, que, aunque a veces se considera compleja o propensa a ambigüedades en otros lenguajes como C++ o Python, en Common Lisp está bien estructurada gracias al uso de métodos genéricos y el algoritmo de resolución de herencia (CPL)¹. Esto permite una mayor flexibilidad en el diseño de jerarquías complejas y favorece una reutilización más eficiente del código [19].

¹El *Class Precedence List* (CPL) es el orden lineal que define cómo se resuelven las herencias en Common Lisp cuando una clase tiene múltiples superclases.

1.5.2. Clases y Notación Prefija

La definición de clases en Common Lisp se realiza mediante la forma especial *defclass*, que sigue consistentemente la notación prefija característica del lenguaje. Esta estructura sintáctica organiza las expresiones con el operador en primera posición, seguido de sus argumentos entre paréntesis.

La sintaxis básica para definir una clase es:

```
(defclass nombre-clase (superclase1 superclase2 ...)  
  ((slot1 :initarg :slot1 :accessor acceso-slot1 ...)  
   (slot2 :initarg :slot2 :accessor acceso-slot2 ...)  
  ...))
```

En esta sintaxis, *nombre-clase* es el símbolo que identifica la clase, mientras que la lista (*superclase1 ...*) especifica la herencia, permitiendo incluso la herencia múltiple. Los *slots* representan los atributos de instancia y pueden incluir opciones como *:initarg*, que define el símbolo utilizado para inicializar el slot al crear instancias, y *:accessor*, que genera automáticamente una función de acceso para el atributo correspondiente [34].

Para la creación de instancias de una clase se realiza mediante el método *make-instance* de la siguiente manera:

```
(make-instance 'nombre-clase :slot1 valor1 :slot2 valor2 ...)
```

La notación prefija característica de Lisp contribuye a una mayor claridad y uniformidad sintáctica en la definición de constructores dentro del lenguaje, facilitando la expresión de estructuras complejas de manera concisa y legible.

En el siguiente apartado se detallan las funciones en Lisp, así como otros mecanismos avanzados que resultan esenciales para la construcción de lenguajes de dominio específico.

1.5.3. Funciones en Lisp

En Common Lisp, las funciones constituyen uno de los elementos fundamentales del lenguaje y su definición es sumamente flexible. Esta flexibilidad se refleja especialmente en la manera en que se gestionan los parámetros de las funciones, permitiendo distintos tipos de argumentos que se adaptan a diversas necesidades. Los principales tipos de parámetros que una función puede recibir son:

- **Parámetros requeridos:** Son los argumentos obligatorios que deben proporcionarse en cada llamada a la función, y su orden es significativo.
- **Parámetros opcionales** (&optional): Estos argumentos no son obligatorios; si no se proporcionan, toman un valor predeterminado (generalmente NIL o un valor especificado). Se indican con el calificador &optional.
- **Parámetros de palabra clave** (&key): Estos argumentos se identifican por su nombre (palabra clave) en lugar de su posición, lo que permite pasar solo los argumentos relevantes sin preocuparse por el orden. Se indican con el calificador &key.
- **Parámetros rest** (&rest): Permiten que una función acepte un número variable de argumentos, que son recogidos en una lista. Esto es útil para funciones que operan sobre una cantidad indefinida de entradas. Se indican con el calificador &rest [16].

Otro aspecto en Common Lisp es el uso de funciones genéricas y métodos, que permiten definir comportamientos especializados y flexibles en la programación orientada a objetos. A continuación, se describen estos conceptos y su relevancia para el diseño del lenguaje propuesto.

1.5.4. Funciones Genéricas y Métodos

Las *funciones genéricas* en Lisp, al igual que las funciones ordinarias, reciben como entrada una lista de argumentos y realizan una serie de operaciones.

Una función genérica define una operación abstracta, especificando su nombre y una lista de parámetros, pero sin implementación [20]. Por ejemplo, si se quisiera definir el área de una figura, se pudiera realizar de la siguiente forma:

```
(defgeneric area (figura))
```

En este caso, el cálculo del área depende del tipo de figura, por lo que no tiene sentido proporcionar una implementación general para dicha operación sin especificar una figura concreta. Precisamente por esto se denominan funciones genéricas: su comportamiento depende de los argumentos que reciben, y la implementación concreta se define para cada caso particular.

Los *métodos* son los encargados de implementar las funciones genéricas. Cada método proporciona una implementación específica de la función genérica para determinadas clases de argumentos, especializando así el comportamiento según los tipos de los parámetros.

Siguiendo con el ejemplo del área de la figura, se pueden definir los siguientes métodos:

```
(defmethod area ((figura circulo))
  (* pi (expt (radio figura) 2)))

(defmethod area ((figura rectangulo))
  (* (base figura) (altura figura)))
```

Cuando se invoca una función genérica, se comparan los argumentos reales con los parámetros de cada uno de sus métodos para encontrar los métodos aplicables, es decir, aquellos cuyos parámetros son compatibles con los argumentos proporcionados en la llamada.

Las funciones genéricas permiten definir métodos especializados en varios parámetros, lo que facilita el polimorfismo múltiple. Además, simplifican la gestión de la herencia múltiple y favorecen la reutilización de código al combinar métodos de forma declarativa.

Por lo anteriormente expuesto, se decidió emplear funciones genéricas para la generación de código en el lenguaje. Esto permite definir, para cada tipo de nodo del AST, la forma en que se produce el código correspondiente.

A continuación, se describen las macros, otro de los mecanismos fundamentales de Lisp que resultan esenciales para la construcción de lenguajes de dominio específico [16].

1.5.5. Macros

Una macro es una función que genera código Lisp: un programa que genera programas [5].

Las macros tienen un funcionamiento diferente al de las funciones. Una función produce un resultado, pero una macro produce una expresión que, al ser evaluada, produce un resultado [17]. Esta conversión se realiza antes de que el código sea evaluado o compilado. El paso que construye la expresión que será evaluada se llama macro-expansión.

Las macros pueden facilitar la escritura y lectura del código al evitar repeticiones, aunque su uso excesivo o poco claro puede dificultar la com-

prensión del programa, especialmente para quienes no estén familiarizados con ellas.

Durante el diseño del lenguaje propuesto en este trabajo, las macros se emplearon para definir nuevas construcciones del lenguaje y transformar expresiones de alto nivel en estructuras más simples, facilitando la generación automática del árbol de sintaxis abstracta y la posterior generación de código.

Un ejemplo de esto puede ser el macro *defclass** que crea una clase y el constructor de la misma clase, utilizando solamente el siguiente código:

```
(defclass* input ()  
  (objeto))
```

En este capítulo se han presentado los conceptos fundamentales que sustentan el desarrollo de este trabajo. Se describieron los principales procesos que motivan la creación de herramientas especializadas para su gestión. Asimismo, se abordaron los fundamentos teóricos de los DSL, los árboles de sintaxis abstracta y las características esenciales de Common Lisp, incluyendo su sistema de objetos, la notación prefija, las funciones genéricas y las macros.

En el siguiente capítulo se explican las decisiones de diseño y las técnicas utilizadas en la implementación del lenguaje desarrollado en este trabajo.

Capítulo 2

Aspectos de las microaplicaciones de gestión

En este capítulo se presentan ejemplos de microaplicaciones de gestión (MAG) extraídos de contextos académicos como la asignación de alumnos ayudantes, la planificación de defensas de tesis y la gestión de la docencia y de ámbitos generales como el control de acontecimientos relevantes y la administración de notas de libros. A partir de estos casos de uso, se identifican los componentes y estructuras que toda MAG presenta: el almacenamiento de información, la definición de consultas y la representación visual de resultados. Finalmente, se realiza una definición de una MAG en lenguaje natural, a partir de sus estructuras básicas.

2.1. Ejemplos de microaplicaciones de gestión

Para definir los principales elementos de un lenguaje orientado al diseño de *microaplicaciones de gestión* (MAG), es importante analizar procesos tanto académicos como de otros ámbitos, que permitan identificar los elementos de estas microaplicaciones. Entre esos procesos se pueden mencionar la asignación de alumnos ayudantes, las defensas de tesis de diploma, la asignación de profesores en las asignaturas del semestre, la gestión de acontecimientos relevantes y la gestión de notas de libros.

A continuación se realiza una descripción de estos procesos.

Asignación de alumnos ayudantes

La asignación de alumnos ayudantes consiste en determinar qué alumno ayudante debe apoyar a qué profesor a impartir una asignatura. De cada alumno se conoce: nombre y grupo. De cada asignatura se conoce: nombre y año al que debe ser impartida. De cada profesor se conoce: nombre y grado científico. Un profesor puede impartir varias asignaturas y tiene asociado a un estudiante por cada asignatura que imparte [41].

En la siguiente tabla se refleja un ejemplo de la asignación de los alumnos ayudantes.

Cuadro 2.1 Asignación de los alumnos ayudantes

Asignatura	Alumno	Profesor
Matemática Discreta	Kevin Manzano	Yudivián Almeida
Estructuras de Datos y Algoritmos	Eduardo Brito	Alberto González
Diseño de Análisis y Algoritmos	Daniel Abad	Rodrigo García
Probabilidades	Jackson Vera	Lázaro González
Matemática Numérica	Marian Álvarez	Fernando Rodríguez

El objetivo de este proceso es facilitar consultas que permitan conocer la relación entre alumnos, profesores y asignaturas. Algunas consultas posibles son:

- ¿Cuántos alumnos ayudantes tiene asignados un profesor determinado?
- ¿Qué asignatura atiende un alumno ayudante y con qué profesor trabaja?
- ¿Qué estudiantes participan en la asignatura *Estructuras de Datos y Algoritmos*?

Este tipo de consultas permite llevar un control sobre las responsabilidades de los alumnos ayudantes y la distribución del trabajo entre los profesores.

Otro proceso que se realiza todos los cursos es la gestión de las defensas de tesis y se describe a continuación.

Defensas de tesis de diploma

La planificación de las defensas de tesis comprende la asignación de los tribunales y la planificación de los horarios y locales para las exposiciones.

Cada exposición está asociada a un estudiante y su tutor, y cuenta con un tribunal conformado por cuatro profesores que desempeñan los roles de oponente, presidente, secretario y vocal [15]. Además, de cada exposición se conoce el horario y el local donde se realizará la defensa. De cada estudiante se conoce: nombre y grupo. De cada profesor se conoce: nombre y grado científico. La planificación debe garantizar que no existan solapamientos en los horarios ni conflictos en la asignación de locales.

En la siguiente tabla se refleja un ejemplo de la planificación de las defensas de tesis tomado de la planificación del curso 2024.

Cuadro 2.2 Planificación de las defensas de tesis

Estudiante	Tutor	Oponente	Presidente	Secretario	Vocal	Fecha	Horario	Local
Marcos Antonio Ochill Trujillo	Lic. Daniel Alejandro Valdés Pérez	Dra. Suilán Estevez Velarde	Dr. Yudivián Almeida Cruz	Lic. Niley González Ferrales	Lic. Roberto García	20 Feb	9:30 am	Aula 5
Víctor Manuel Amador Sosa	Dra. Suilán Estevez Velarde	Lic. Roberto García	Dr. Yudivián Almeida Cruz	Lic. Daniel Alejandro Valdés Pérez	Lic. Alejandro Beltrán Varela	20 Feb	10:30 am	Aula 5
Anabel Benítez González	MSc. Fernando Rodríguez Flores	Lic. Manuel Vila Valiente	Lic. David Guaty	Lic. Alejandra Monzón Peña	Lic. Lázaro Daniel González Martínez	24 Feb	12:30 pm	Aula 9

El objetivo de este proceso es permitir la realización de consultas como:

- ¿Qué estudiantes defienden su tesis en una fecha determinada?
- ¿Qué horarios tiene libre un profesor determinado?
- ¿En qué local y horario se realizará la defensa de un estudiante?
- ¿Cuántas defensas tiene asignado un profesor como miembro del tribunal?

Estas consultas facilitan la organización y el seguimiento de la planificación de las defensas de tesis. Otro proceso es la asignación de los profesores en las asignaturas del semestre.

Asignación de docencia

La gestión de la docencia consiste en asignar un conjunto de profesores a un conjunto de asignaturas que van a impartir durante un semestre, tanto

en la modalidad de conferencias como en clases prácticas. De cada profesor se conoce: nombre y grado científico. De cada asignatura: nombre, carrera y año en que se imparte. Un profesor puede impartir varias asignaturas, incluso varias modalidades dentro de la misma asignatura.

En la siguiente tabla se refleja un ejemplo de la planificación de la docencia.

Cuadro 2.3 Planificación de la docencia

Asignatura	Profesor	Modalidad	Carrera
Matemática Discreta	Rodrigo García	Conferencia	Ciencia de la Computación
Análisis Matemático	Ernesto López	Clase Práctica	Ciencias de Datos
Matemática Numérica	Fernando Rodríguez	Conferencia	Matemática

El objetivo de este proceso es permitir la realización de consultas sobre docencia asignada a los profesores, como:

- ¿Qué profesor imparte la asignatura *Matemática Discreta*?
- ¿Cuáles asignaturas imparte el profesor *Ernesto López*?
- ¿Qué profesores imparten asignaturas en la modalidad de *conferencia*?
- ¿Qué profesores participan en la docencia durante el semestre?

Estas consultas permiten dar seguimiento a la planificación docente e identificar sobrecargas o vacíos en docencia de los profesores. Otro proceso es la gestión de acontecimientos relevantes.

Acontecimientos relevantes

La gestión de acontecimientos y fechas relevantes consiste en registrar, organizar y consultar hechos significativos dentro del ámbito académico, profesional o personal, tales como aniversarios, hitos históricos, descubrimientos, nacimientos y efemérides importantes. Esto es relevante porque garantiza que no se pasen por alto sucesos clave y facilita la planificación de actividades futuras.

En la facultad de Matemática y Computación, por ejemplo, se suele resaltar el Día de la Matemática en Cuba, el Día de la Mujer Matemática, o el Día de las Niñas en las las Tecnologías de la Información y la Comunicación (TIC). Conocer con anterioridad, qué fechas relevantes se acercan, permitiría planificar con tiempo actividades relacionadas con estos motivos.

Este proceso implica almacenar cada acontecimiento junto con su fecha, una descripción y un conjunto de etiquetas temáticas que permiten clasificarlo y facilitar su recuperación.

En la siguiente tabla se refleja un ejemplo de la gestión de acontecimientos relevantes.

Cuadro 2.4 Ejemplos de acontecimientos relevantes

Acontecimiento	Fecha	Etiquetas	Descripción
Día de la Matemática en Cuba	31-10	matemática, Cuba	Fecha en que se constituyó la Sociedad Cubana de Matemática en 1978. Posteriormente, en 1989, esta sociedad pasó a denominarse Sociedad Cubana de Matemática y Computación.
Día de la Mujer Matemática	12-05	mujeres matemáticas, igualdad de género	Celebración instaurada en 2018 por el Comité de Mujeres y Matemáticas de las Tecnologías de la Información y la Comunicación (IMU) en honor al nacimiento de Maryam Mirzakhani, primera mujer galardonada con la Medalla Fields, para visibilizar su legado.
Día de las Niñas en las TIC	Cuarto jueves de abril	TIC, género, inclusión digital	Iniciativa de la Unión Internacional de Telecomunicaciones (UIT) desde 2011 para fomentar la participación de niñas y jóvenes en las Tecnologías de la Información y la Comunicación, reduciendo la brecha digital de género.

El objetivo de este proceso es permitir la realización de consultas útiles para la gestión de acontecimientos relevantes, tales como:

- ¿Qué acontecimientos relevantes ocurren en una fecha o rango de fechas determinado?
- ¿Qué efemérides están asociadas a una etiqueta temática específica?

- ¿Cuáles son los acontecimientos relacionados con un área temática concreta, como computación o literatura?
- ¿Qué eventos históricos o aniversarios importantes se celebran en fechas cercanas?

Estas consultas facilitan la organización, el seguimiento y la recuperación eficiente de información sobre hechos significativos, permitiendo planificar actividades y recordar fechas importantes.

Otro proceso es la gestión de las notas de libros, que se describe a continuación.

Notas de libros

La gestión de notas de libros consiste en administrar un conjunto de anotaciones etiquetadas, permitiendo realizar consultas sobre ellas.

En este proceso, cada nota debe contener no solo la referencia al libro y las etiquetas asociadas, sino también el texto específico de la cita o anotación realizada. De esta forma, se almacena la información relevante que el usuario desea conservar o consultar posteriormente. Por ejemplo, una nota puede incluir un fragmento destacado del libro, una reflexión personal o una cita textual, permitiendo así un registro más completo y útil para futuras búsquedas y análisis.

En la siguiente tabla se refleja un ejemplo de la gestión de notas de libros.

Cuadro 2.5 Gestión de notas de libros

Nota	Libro	Etiquetas
"El secreto de una buena vejez no es otra cosa que un pacto honrado con la soledad."	Cien años de soledad	Realismo mágico, Literatura latino-americana
"Si quieres mantener un secreto, debes también ocultarlo de ti mismo."	1984	Distopía, Política

El objetivo es poder realizar consultas que permitan buscar notas por etiquetas y libros.

Los procesos descritos anteriormente comparten estructuras y necesidades comunes que justifican la creación de herramientas especializadas

para su gestión. Esta observación motiva a identificar los elementos comunes presentes en las microaplicaciones de gestión. En la sección siguiente, se analiza en detalle dichos elementos.

2.2. Elementos de las microaplicaciones de gestión

Para el diseño de las microaplicaciones de gestión se distinguen tres componentes principales: el almacenamiento de la información, la definición de las consultas y la representación visual de la microaplicación. Cada una de ellas se analiza en detalle en las secciones siguientes.

2.2.1. Almacenamiento de la información

Este componente responde a la pregunta: ¿cómo se almacenan los datos en las microaplicaciones de gestión?

Al revisar los procesos descritos en la sección anterior, se identifica que cada uno involucra *entidades* clave. Se le llama *entidad* a cualquier objeto que agrupa múltiples atributos. Por ejemplo:

- En la asignación de alumnos ayudantes, las entidades son:
 - *Estudiantes*: contiene los atributos nombre y grupo.
 - *Asignaturas*: contiene los atributos nombre y año.
 - *Profesores*: contiene los atributos nombre y grado científico.
 - *Asignaciones*: contiene un estudiante, un profesor y una asignatura.
- En las defensas de tesis de diploma, las entidades son:
 - *Estudiantes*: contiene los atributos nombre y grupo.
 - *Profesores*: contiene los atributos nombre y grado científico.
 - *Planificaciones*: contiene un estudiante, profesores identificados como tutor, oponente, presidente, secretario y vocal, y parámetros adicionales como fecha, horario y local.

El almacenamiento de la información se logra creando una colección o conjunto de datos para cada entidad identificada.

A continuación, se describen las componentes restantes que completan el diseño de estas aplicaciones: la definición de las consultas y la representación visual de la microaplicación.

2.2.2. Definición de las consultas

Otro componente de las MAG es la definición de las consultas.

Un ejemplo de consulta en el contexto de la *Asignación de Alumnos Ayudantes*, es obtener el nombre de los estudiantes que son atendidos por el profesor “Juan Cruz” y que pertenecen al grupo “C412”.

Para ello, se recorre cada elemento de las entidades Estudiantes y Asignaciones y se verifica que el atributo profesor en la entidad Asignaciones sea igual a “Juan Cruz” y que el atributo grupo en la entidad Estudiantes sea igual a “C412”. Finalmente, se muestra el atributo nombre de la entidad Estudiantes como resultado.

Una consulta se compone de los siguientes elementos (se indica entre paréntesis qué representan en el ejemplo anterior):

- **Selección de resultados:** establece los atributos que serán devueltos como resultado de la consulta (por ejemplo, mostrar el atributo nombre de la entidad Estudiantes).
- **Acceso a atributos:** selección de los campos o propiedades de las entidades sobre los que se desea filtrar o proyectar información (por ejemplo, acceder al atributo profesor de la entidad Asignaciones y a los atributos nombre y grupo de la entidad Estudiantes).
- **Iteraciones de entidades:** expresa en qué entidades se debe buscar la información (por ejemplo, recorrer cada elemento de las entidades Estudiantes y Asignaciones).
- **Operadores lógicos:** uso de conectores tales como and, or y not para combinar múltiples cláusulas (por ejemplo, uso del operador and para combinar las condiciones sobre el profesor y el grupo).
- **Operadores de comparación:** definición de relaciones de igualdad (=) y desigualdad (!=) entre atributos y valores (por ejemplo, comparar si el atributo profesor es igual a “Juan Cruz” y si el atributo grupo es igual a “C412”).
- **Valores atómicos:** literales, como cadenas de texto, que se emplean en las expresiones de comparación (por ejemplo, los literales “Juan Cruz” y “C412”).

Cada consulta en las microaplicaciones de gestión está asociada a una vista donde se muestran los resultados de dicha operación.

En la siguiente sección se describen los componentes visuales que permiten la interacción con las MAG.

2.2.3. Representación visual de una microaplicación

En este componente se definen los elementos que conforman la interfaz de usuario de la microaplicación de gestión. Estos elementos visuales no solo organizan la información, sino que también facilitan la interacción del usuario con las consultas y resultados. A continuación se describen los elementos visuales que se pueden considerar en una MAG, ilustrados con ejemplos de las MAG presentadas:

- **Página:** Es la vista o pantalla de una microaplicación, que funciona como un contenedor que organiza un conjunto de elementos visuales.

Por ejemplo, en la **Asignación de Alumnos Ayudantes**, podría haber una página dedicada a la “Asignación de Alumnos Ayudantes por Asignatura” y otra para “Consultar Asignaciones de un Profesor”. De manera similar, en las **Defensas de Tesis**, existiría una página principal para la “Planificación de Defensas” y otra para “Consultar las fechas y horarios libres de los profesores que comprenden tribunales”.

El concepto de *página* es abstracto y su significado concreto depende del formato de salida en que se implemente la MAG. Por ejemplo, en una aplicación web desarrollada con Django, una página correspondería a una página web específica. En cambio, en una aplicación de consola, una página podría representarse como un bloque de información que se muestra al usuario en la terminal.

- **Lista ordenada:** Un conjunto numerado de elementos o pasos. Podría utilizarse, por ejemplo, para enumerar los miembros de un tribunal en un formato de lista.
- **Input:** Son los campos de entrada de datos que permiten al usuario entrar información a la aplicación.

En la **Asignación de Alumnos Ayudantes**, un input permitiría al usuario escribir el “nombre de un profesor” para buscar sus asignaciones, o el “nombre de una asignatura” para ver los estudiantes que la imparten.

En el caso de los **Acontecimientos Relevantes**, un input de fecha o un input de texto para las etiquetas temáticas serían cruciales para realizar esas búsquedas.

- **Elemento de lista:** elementos individuales dentro de una lista, que suelen ser los resultados de una consulta.

Por ejemplo, si un usuario consulta las asignaturas que imparte un profesor en la **Asignación de Docencia**, cada asignatura mostrada (*Matemática Discreta*, *Análisis Matemático*, etc.) sería un elemento de lista.

En las **Notas de Libros**, cada anotación recuperada (“El secreto de una buena vejez...”, “Si quieres mantener un secreto...”) se pudiera representar como un elemento de lista.

- **Título:** Encabezado principal o secundario que identifica la sección o el contexto de los datos mostrados.

Cada una de las tablas presentadas en los ejemplos de MAG, como “Asignación de los alumnos ayudantes” o “Planificación de las defensas de tesis”, se presentarían en la interfaz con un título claro que las identifique. Además, las secciones dentro de una página, como “Consultas Rápidas” o “Detalles del Acontecimiento”, también tendrían títulos.

- **Texto:** Bloques de texto o etiquetas informativas que proporcionan descripciones, instrucciones o contextualización.

Por ejemplo, debajo de un input de búsqueda, podría haber un texto que indique “Ingrese el nombre del profesor para ver sus asignaciones”. De igual manera, las descripciones detalladas de los acontecimientos en **Acontecimientos Relevantes** o los fragmentos de las notas en **Notas de Libros** se mostrarían como bloques de texto.

Todas las MAG cuentan con una página principal desde la cual se puede seleccionar la consulta que se desea realizar, actuando como un panel de control inicial para la navegación del usuario.

Teniendo en cuenta todos estos elementos, en la siguiente sección se describe un ejemplo de la MAG de Asignación de Alumnos Ayudantes a través de los componentes de representación de los datos, definición de consultas y representación visual.

2.3. Descripción de una microaplicación de gestión

La microaplicación de gestión (MAG) para la asignación de alumnos ayudantes puede definirse, a partir de sus componentes principales, de la siguiente manera:

Representación de los datos

■ Entidades:

- Estudiante con atributos: nombre, grupo.
- Asignatura con atributos: nombre, año.
- Profesor con atributos: nombre, grado_científico.
- Asignación que relaciona a los estudiantes, asignaturas y profesores.

Definición de las consultas

1. Listar todas los estudiantes con su grupo.

Para ello, se recorre cada elemento de la entidad Estudiantes y se muestra los atributos nombre y grupo de la entidad Estudiantes como resultado.

Esta consulta se puede describir con los siguientes elementos:

- **Selección de los resultados:** Mostrar los atributos nombre y grupo de la entidad Estudiantes.
- **Acceso a atributos:**
 - Acceso a atributo nombre de la entidad Estudiantes.
 - Acceso a atributo grupo de la entidad Estudiantes.
- **Iteraciones de entidad:** Iterar sobre la entidad Estudiantes.

2. Obtener el nombre de los estudiantes del grupo "C412" que son atendidos por un profesor.

Para ello, se recorre cada elemento de las entidades Estudiantes y Asignaciones y se verifica que el atributo profesor en la entidad Asignaciones sea igual al nombre recibido en un input como por ejemplo "Juan Cruz" y que el atributo grupo en la entidad Estudiantes sea igual a "C412". Finalmente, se muestra el atributo nombre de la entidad Estudiantes como resultado.

Esta consulta se puede describir con los siguientes elementos:

- **Selección de los resultados:** Mostrar el atributo nombre de la entidad Estudiantes.
- **Acceso a atributos:**
 - Acceso al atributo profesor de la entidad Asignaciones.
 - Acceso al atributo estudiante de la entidad Asignaciones.
 - Acceso al atributo grupo de la entidad Estudiantes.
 - Acceso al atributo nombre de la entidad Estudiantes.
- **Iteraciones de entidad:** Iterar sobre las entidades Estudiantes y Asignaciones.
- **Operadores lógicos:** Utilizar el operador lógico and para combinar las condiciones sobre el profesor y el grupo.
- **Operadores de comparación:** Comparaciones de igualdad (=) entre atributos y valores atómicos.
- **Valores atómicos:** Valores como “Juan Cruz” para el nombre del profesor y “C412” para el grupo.

3. Obtener los nombres de los estudiantes que imparten una asignatura.

Para ello, se recorre cada elemento de la entidad Asignaciones y se verifica que el atributo asignatura en la entidad Asignaciones sea igual al nombre recibido en un input como por ejemplo “Estructuras de Datos y Algoritmos”. Finalmente, se muestra el atributo estudiante de la entidad Asignaciones como resultado.

Esta consulta se puede describir con los siguientes elementos:

- **Selección de los resultados:** Mostrar el atributo estudiante de la entidad Asignaciones.
- **Acceso a atributos:**
 - Acceso a atributo asignatura de la entidad Asignaciones.
 - Acceso a atributo estudiante de la entidad Asignaciones.
- **Iteraciones de entidad:** Iterar sobre la entidad Asignaciones.
- **Operadores de comparación:** Comparaciones de igualdad (=) entre atributos y valores atómicos.
- **Valores atómicos:** Valor “Estructuras de Datos y Algoritmos” para el nombre de la asignatura.

Representación visual de la microaplicación:**■ Página principal:**

- **Título:** “MAG de Asignación de Alumnos Ayudantes”
- **Texto:** Breve descripción de la microaplicación y sus funcionalidades principales.
- **Lista de opciones:**
 - Elemento de lista: “Consultar listado de estudiantes y sus grupos”
 - Elemento de lista: “Consultar estudiantes atendidos por un profesor”
 - Elemento de lista: “Consultar estudiante que imparte una asignatura”
- **Input:** Barra de búsqueda para filtrar consultas o seleccionar la opción deseada.

■ Página de consulta 1:

- **Título:** “Listado de estudiantes y sus grupos”
- **Texto:** Instrucción: “A continuación se muestran todos los estudiantes junto a su grupo.”
- **Lista de resultados:** Para cada estudiante, un elemento de lista con el nombre y el grupo correspondiente.

■ Página de consulta 2:

- **Título:** “Estudiantes atendidos por un profesor”
- **Texto:** Instrucción: “Ingrese el nombre del profesor para ver los estudiantes asignados.”
- **Input:** Campo de texto para introducir el nombre del profesor.
- **Lista de resultados:** Elementos de lista con los nombres de los estudiantes atendidos por el profesor consultado.

■ Página de consulta 3:

- **Título:** “Estudiante que imparte una asignatura”
- **Texto:** Instrucción: “Ingrese el nombre de la asignatura para ver el estudiante asignado.”

- **Input:** Campo de texto para introducir el nombre de la asignatura.
- **Lista de resultados:** Elemento de lista con el nombre del estudiante que imparte la asignatura consultada.

En este capítulo se han presentado diversos ejemplos de microaplicaciones de gestión, identificando sus componentes fundamentales: el almacenamiento de información, la definición de consultas y la representación visual de los resultados. Esta caracterización servirá de base para la formalización y construcción de un lenguaje para la descripción de microaplicaciones de gestión.

Capítulo 3

Aspectos del lenguaje propuesto

En este capítulo se describen los aspectos fundamentales del lenguaje propuesto para la representación y consulta de datos en el contexto de las microaplicaciones de gestión (MAG). Se presentan las estructuras principales del lenguaje, así como las macros y clases diseñadas para facilitar la definición de entidades, consultas y componentes visuales. El objetivo es proporcionar una visión detallada de los elementos que conforman el lenguaje, su sintaxis y su funcionalidad, mostrando ejemplos prácticos que ilustran su uso y ventajas en la construcción de aplicaciones orientadas a la gestión de información. Finalmente, se diseña el ejemplo descrito en el capítulo anterior utilizando el lenguaje propuesto en este trabajo.

3.1. Abstracción y generación automática de clases en el lenguaje

El lenguaje presentado en este trabajo está dedicado a abstraer los componentes de las MAG tratados en el capítulo anterior. Para ello, se crearon las siguientes estructuras principales:

- Declaración de entidades
- Declaración de instancia de una entidad
- Declaración de acceso a atributo
- Declaración de operadores booleanos (and, or y not)

- Declaración de operadores de comparación (igualdad y desigualdad)
- Declaración de iteraciones
- Definición de consultas

Además, para la representación visual se crearon las siguientes estructuras:

- Definición de página
- Definición de lista ordenada
- Definición de ítem
- Definición de título
- Definición de literales como la cadena de texto (*string*)

Dado que se está utilizando Lisp, para construir el AST únicamente se necesita la definición de la clase correspondiente al nodo y una función constructora. En Common Lisp, esto suele hacerse por separado: por un lado se define la clase y por otro el constructor. Por ejemplo:

```
(defclass nodo ()  
  ((valor :accessor valor  
         :initarg :valor)))  
  
(defun crear-nodo (&key valor)  
  (make-instance 'nodo :valor valor))
```

Sin embargo, este patrón se repite constantemente: una definición de clase y su constructor asociado. Aprovechando las capacidades de abstracción de Lisp, es posible automatizar este proceso mediante un macro. Precisamente, esa es la idea detrás del macro `defclass*`, que se creó con el objetivo de definir la clase y, al mismo tiempo, generar automáticamente un constructor por defecto. Esta macro solo necesita un nombre para la clase, una referencia a la clase de la cual hereda y una lista de atributos. Además, al tratarse de un macro, se puede añadir como parámetros adicionales: el nombre de dicho constructor (`:constructor-name`) y la forma de los argumentos del constructor, esta última mediante la opción (`:lambda-list`).

Por ejemplo, si se quisiera crear una clase llamada **ejemplo**, con los atributos nombre y cuerpo, el código quedaría de la siguiente manera:

```
(defclass* ejemplo ()  
  (elementos)  
  :constructor-name ejemplo  
  :lambda-list (&rest elementos))
```

Con el código anterior se generaría el siguiente código:

```
(defclass ejemplo ()  
  (elementos :accessor elementos :initarg :elementos))  
  
(defun new-ejemplo (&rest elementos)  
  (make-instance 'clase-ejemplo :elementos elementos))
```

Para cada una de las estructuras del lenguaje propuesto, se definió una clase que contiene toda la información necesaria para describir los elementos de las MAG. Utilizando los constructores, convenientemente anidados, de estas clases, se pueden construir consultas o componentes visuales para la microaplicación.

Suponiendo que el constructor de la página se llame `pagina` y el constructor del título se llame `titulo`, una página se podría describir de la siguiente forma:

```
(pagina (titulo "Página de ejemplo")  
  "Esto es solo una página con un texto")
```

En este ejemplo, la función `pagina` devuelve una instancia de página cuyos elementos son: una instancia de `titulo` y una instancia de `texto`, que representa un texto simple colocado directamente en la página.

Para definir el lenguaje solo se necesitan los constructores de cada uno de los elementos identificados, a lo cual se dedicarán las siguientes secciones.

3.2. Elementos del lenguaje

En esta sección se describen los elementos fundamentales del lenguaje. Cada clase modela una estructura específica del lenguaje, facilitando la representación y manipulación de entidades, consultas, operadores y componentes visuales.

En esta sección, primero se describirán los componentes para la representación visual, luego los elementos para la representación de datos y finalmente los elementos para la definición de consultas.

3.2.1. Definición de texto

Entre los elementos más sencillos de la representación visual se encuentra el *texto*. Para representar un texto se definió la clase **texto**. Esta clase posee como propiedad la cadena de texto que se quiere mostrar.

El usuario puede definir un texto en el lenguaje propuesto de la siguiente forma:

```
(text "31 de Octubre")
```

3.2.2. Definición de título

Para representar un *título* se definió la clase **título**. Esta clase posee como propiedad solamente la información que se va destacar.

El usuario puede definir un título en el lenguaje propuesto de la siguiente forma:

```
(titulo "Menú de consultas")
```

3.2.3. Definición de entrada de datos

Para representar una *entrada de datos* se definió la clase **input**. Esta clase posee como propiedad el objeto o expresión que se quiere consultar o filtrar.

El usuario puede definir una entrada de datos en el lenguaje propuesto de la siguiente forma:

```
(input "Ingrese el nombre de un estudiante")
```

3.2.4. Definición de elemento de listas

Para representar un *elemento* de una lista se definió la clase **item**. Esta clase posee como propiedad una serie de elementos que conforman el contenido del ítem.

El usuario puede definir un elemento de lista en el lenguaje propuesto de la siguiente forma:

```
(item "Pablo Martínez" "14 de Noviembre")
```

3.2.5. Definición de una página

Para representar una página se definió la clase **pagina** que posee como propiedad una lista de elementos.

El usuario puede definir una página en el lenguaje propuesto de la siguiente forma:

```
(pagina
  elemento-1
  elemento-2
  ...
  elemento-n)
```

donde los símbolos `elemento-1`, `elemento-2`, ..., `elemento-n` representan los distintos elementos que se desean visualizar en la página. Un ejemplo concreto, utilizando las clases presentadas anteriormente, podría ser el siguiente:

```
(pagina
  (titulo "Menú de consultas")
  (lista
    (item "1. Lista los acontecimientos relevantes en un
          rango de fechas")
    (item "2. Lista las efemérides asociadas a una etiqueta
          determinada")
    (item "3. Lista los acontecimientos relevantes que se
          celebran en Octubre"))
  (input "Seleccione una opción: "))
```

3.2.6. Definición de entidades

En el contexto del lenguaje propuesto, una *entidad* se define como un objeto que agrupa múltiples atributos. Para representar una entidad se definió la clase **entidad**. Esta clase posee como propiedades el nombre de la entidad y una lista de atributos que la componen.

El usuario puede definir una entidad en el lenguaje propuesto de la siguiente forma:

```
(entidad nombre-entidad atributo1 atributo2 ... atributoN)
```

Por ejemplo, si se desea definir una entidad estudiantes que tiene como atributos nombre y grupo, se puede definir la entidad de la siguiente manera:

```
(entidad "Estudiantes" nombre grupo)
```

En el caso que una entidad tenga un atributo que representa a un atributo de otra entidad se debe definir como una llave foránea. Por ejemplo, si se desea definir una entidad Asignaciones que tiene como atributos profesor, estudiante y asignatura, donde cada uno es una clave foránea que hace referencia al atributo nombre de las entidad Profesores, Estudiantes y Asignaturas, respectivamente; se puede definir la entidad Asignaciones de la siguiente manera:

```
(entidad "Asignaciones"
  (profesor :foreign-key "Profesores" nombre)
  (estudiante :foreign-key "Estudiantes" nombre)
  (asignatura :foreign-key "Asignaturas" nombre))
```

3.2.7. Definición de una instancia de una entidad

Para representar una instancia de una entidad se definió la clase **instancia-entidad**. Esta clase posee como propiedad el nombre de la entidad a la que pertenece la instancia.

El usuario puede definir una instancia de entidad en el lenguaje propuesto de la siguiente forma:

```
(instancia-de "Estudiantes")
```

3.2.8. Declaración de acceso a un atributo de una entidad

Para representar un acceso a un atributo de una entidad se definió la clase **acceso-a-atributo**. Esta clase posee como propiedades el nombre del atributo, la entidad a la que hace referencia el atributo y una instancia de la entidad.

Como por cada entidad, se define un conjunto de atributos, se debería definir un acceso a cada uno de los atributos de la entidad. Por eso para

cada combinación de atributo e instancia, se creará una instancia de la clase **acceso-a-atributo**. El usuario puede definir un acceso a un atributo de una entidad en el lenguaje propuesto de la siguiente forma:

```
(nombre-del-atributo-de-la-entidad
  instancia-entidad)
```

Por ejemplo, si se desea acceder a los atributos `nombre` y `grado-cientifico` de una instancia de la entidad `Profesores` se puede definir de la siguiente manera:

```
(nombre-del-profesor instancia-profesor)
(grado-del-profesor instancia-profesor)
```

Aprovechando las capacidades de abstracción de Lisp, todos estos accesos a atributos se pueden definir de forma automática mediante la macro *def-entidad* que creará una función para cada acceso a atributo de la entidad. Por lo que ahora, se puede definir una entidad y sus accesos a atributos de la siguiente manera:

```
(def-entidad profesor nombre grado-cientifico)
```

3.2.9. Declaración de operadores de comparación

Entre las operaciones de comparación presentes en el lenguaje se encuentran las *operaciones de igualdad y desigualdad* (`==`, `!=`). Para permitir al usuario el uso de las operaciones presentadas fue necesario la existencia de una clase para cada uno de estos operadores.

Para representar el operador de comparación de igualdad (`==`) se definió la clase **comparacion-igualdad**. Esta clase posee como propiedades los elementos izquierdo y derecho de la comparación.

El usuario puede definir un operador de comparación de igualdad en el lenguaje propuesto de la siguiente forma:

```
(es-igual (presidente-de-la-defensa profesor) "Juan Pérez")
```

Para representar el operador de comparación de desigualdad (`!=`) se definió la clase **comparacion-desigualdad**. Esta clase posee como propiedades los elementos izquierdo y derecho de la comparación.

El usuario puede definir un operador de comparación de desigualdad en el lenguaje propuesto de la siguiente forma:

```
(no-es-igual (secretario-de-la-defensa profesor) "María López")
```

3.2.10. Declaración de operadores lógicos

Entre las *operaciones lógicas* presentes en el lenguaje se encuentran (and, or, not). Para permitir al usuario el uso de las operaciones presentadas fue necesario la existencia de una clase para cada uno de estos operadores.

Para representar el operador de negación (*not*) se definió la clase **operador-not**. Esta clase posee como propiedad la cláusula a negar.

El usuario puede definir un operador de negación en el lenguaje propuesto de la siguiente forma:

```
(no (es-igual (fecha-de-la-defensa defensa) "31 de Octubre"))
```

Para representar el operador de disyunción lógica (*or*) se definió la clase **operador-or**. Esta clase posee como propiedad las cláusulas a evaluar.

El usuario puede definir un operador de disyunción lógica en el lenguaje propuesto de la siguiente forma:

```
(se-cumple-una-de-estas
  (es-igual (fecha-de-la-defensa defensa) "20 de Junio")
  (es-igual (fecha-de-la-defensa defensa) "23 de Junio")
  (es-igual (fecha-de-la-defensa defensa) "24 de Junio"))
```

Para representar el operador de conjunción lógica (*and*) se definió la clase **and-operator**. Esta clase posee como propiedad solamente las cláusulas a evaluar.

El usuario puede definir un operador de conjunción lógica en el lenguaje propuesto de la siguiente forma:

```
(se-cumple-simultaneamente
  (es-igual (libro-de-la-nota nota) "Cien años de soledad")
  (es-igual (etiqueta-de-la-nota nota) "Realismo mágico"))
```

Dado que los constructores de las clases, que constituyen los elementos del lenguaje, son funciones de Lisp, y la definición de funciones en este lenguaje es flexible, es posible disponer de múltiples formas para crear una misma instancia. En este sentido, por ejemplo, se puede crear una instancia de un nodo *and* utilizando la función *al-mismo-tiempo*:

```
(al-mismo-tiempo
  (es-igual (libro-de-la-nota nota) "Cien años de soledad")
  (es-igual (etiqueta-de-la-nota nota) "Realismo mágico"))
```

3.2.11. Definición de consultas

En el lenguaje propuesto una consulta está compuesta por:

- un nombre,
- una lista de argumentos,
- una lista de iteraciones que expresa sobre qué entidades hay que recorrer para obtener el resultado
- una pregunta o filtrado para obtener el resultado, y
- un resultado que se quiere mostrar

Por ejemplo, si se tiene una consulta que sea obtener todos los profesores que tengan grado científico Dr, el usuario puede definir la consulta en el lenguaje propuesto de la siguiente forma:

```
(def-consulta profesores_con_grado_dr
  :argumentos ()
  :iteraciones ((profesor profesores))
  :que-mostrar ((nombre-del-profesor profesor))
  :pregunta (es-igual (grado-del-profesor profesor) "Dr"))
```

Otro ejemplo de una consulta podría ser en la *Asignación de Alumnos Ayudantes*, obtener todos los estudiantes atendidos por un profesor que están en el grupo "C412":

```
(def-consulta estudiantes_atendidos_por_profesor
  :argumentos (profesor_nombre)
  :iteraciones ((estudiante estudiantes)
                (asignacion asignaciones))
  :que-mostrar ((nombre-del-estudiante estudiante))
  :pregunta (se-cumple-simultaneamente
             (es-igual
              (profesor-de-la-asignacion asignacion)
              profesor_nombre))
```

```
(es-igual
  (estudiante-de-la-asignacion asignacion)
  estudiante)
(es-igual
  (grupo-del-estudiante estudiante)
  "C412"))
```

3.3. Descripción de una microaplicación de gestión en el lenguaje propuesto

A continuación se muestra cómo expresar en el lenguaje propuesto la microaplicación de gestión para la asignación de alumnos ayudantes, siguiendo la estructura y componentes descritos previamente.

Definición de entidades:

```
(def-entidad estudiante nombre grupo)
(def-entidad asignatura nombre año)
(def-entidad profesor nombre grado-cientifico)
(def-entidad asignacion
  (profesor :foreign-key profesores nombre)
  (estudiante :foreign-key estudiantes nombre)
  (asignatura :foreign-key asignaturas nombre))
```

Definición de consultas:

1. Listar todos los estudiantes con su grupo:

```
(def-consulta listado_de_estudiantes
  :iteraciones ((estudiante estudiantes))
  :que-mostrar ((nombre-del-estudiante estudiante)
                (grupo-del-estudiante estudiante)))
```

2. Obtener el nombre de los estudiantes del grupo "C412" que son atendidos por un profesor:

```
(def-consulta estudiantes_atendidos_por_profesor
  :argumentos (profesor_nombre))
```

```

:iteraciones ((estudiante estudiantes)
              (asignacion asignaciones))
:que-mostrar ((nombre-del-estudiante estudiante))
:pregunta (se-cumple-simultaneamente
           (es-igual
            (profesor-de-la-asignacion asignacion)
            profesor_nombre)
           (es-igual
            (estudiante-de-la-asignacion asignacion)
            estudiante)
           (es-igual
            (grupo-del-estudiante estudiante)
            "C412")))

```

3. Obtener el nombre del estudiante que imparte una asignatura:

```

(def-consulta estudiante_imparte_asignatura
  :argumentos (asignatura_nombre)
  :iteraciones ((asignacion asignaciones))
  :que-mostrar ((estudiante-de-la-asignacion asignacion))
  :pregunta (es-igual
            (asignatura-de-la-asignacion asignacion)
            asignatura_nombre))

```

Una vez definidas las entidades y las consultas del lenguaje propuesto, se procede a construir la interfaz visual de la microaplicación.

Representación visual de la microaplicación

Página principal:

```

(page main
  (title "MAG de Asignación de Alumnos Ayudantes")
  (text "Microaplicación para gestionar la asignación de estudiantes
        como ayudantes en distintas asignaturas. Permite consultar los
        estudiantes atendidos por un profesor y el estudiante que imparte
        una asignatura.")
  (lista
    (link-item "Listado de estudiantes y sus grupos")
    (link-item "Consultar estudiantes atendidos por un profesor")
    (link-item "Consultar estudiante que imparte una asignatura"))
  (input "Filtre o seleccione una opción: "))

```

Página de consulta 1:

```
(page mostrar-estudiantes
  (title "Listado de estudiantes y sus grupos")
  (text "A continuación se muestran todos los estudiantes junto a
        su grupo.")
  (lista
    ;; Aquí se mostraría el resultado de la consulta
    ;; mostrar-estudiantes
  ))
```

Página de consulta 2:

```
(page estudiantes-atendidos-por-profesor
  (title "Estudiantes atendidos por un profesor")
  (text "Ingrese el nombre del profesor para ver los estudiantes
        asignados.")
  (input "Nombre del profesor: ")
  (lista
    ;; Aquí se mostrarían los resultados de la consulta
    ;; estudiantes-atendidos-por-profesor
  ))
```

Página de consulta 3:

```
(page estudiante-imparte-asignatura
  (title "Estudiante que imparte una asignatura")
  (text "Ingrese el nombre de la asignatura para ver el estudiante
        asignado.")
  (input "Nombre de la asignatura: ")
  (lista
    ;; Aquí se mostraría el resultado de la consulta
    ;; estudiante-imparte-asignatura
  ))
```

En este capítulo se presentó el lenguaje propuesto para microaplicaciones de gestión (MAG), detallando sus estructuras fundamentales para modelar datos (entidades, atributos, instancias), construir consultas (operadores lógicos, comparaciones, iteraciones) y diseñar interfaces visuales (páginas, listas, textos).

Capítulo 4

Generación de código

En el presente capítulo se muestra el mecanismo utilizado para obtener las microaplicaciones de gestión (MAG) representadas mediante los árboles de sintaxis abstracta en formatos como Consola de Python, aplicaciones web en Django o sistemas de gestión de bases de datos como MySQL.

En el lenguaje propuesto el usuario puede seleccionar a qué formato desea exportar el código de las MAG. Para esto, durante el diseño del lenguaje se definió para la generación de código un conjunto de funciones que especifican cómo se traduce cada nodo del AST en cada formato de salida.

Como se mencionó en el capítulo 3 los posibles elementos del AST para la representación de los datos, la definición de consultas y la visualización de los datos son:

- Declaración de entidades
- Declaración de instancia de una entidad
- Declaración de acceso a atributo
- Declaración de operadores booleanos (and, or y not)
- Declaración de operadores de comparación (igualdad y desigualdad)
- Declaración de iteraciones
- Definición de consultas
- Definición de página
- Definición de lista ordenada

- Definición de ítem
- Definición de título
- Definición de literales como la cadena de texto (*string*)

Al usar convenientemente los constructores de las clases que representan los nodos del AST se obtiene el árbol que representa la MAG descrita. A partir de este árbol es posible obtener una representación de la MAG en cualquiera de los formatos definidos en la herramienta. Para ello se debe recorrer cada uno de los nodos, describiendo ese nodo en el formato deseado. Para ello se usa la información almacenada en cada nodo.

En la siguiente sección, se presentará en detalle la arquitectura de clases abstractas y métodos que soportan la generación de código en los diferentes formatos. Se explicará cómo estas clases y funciones permiten traducir los elementos del lenguaje a representaciones específicas para cada entorno de exportación.

Al finalizar el capítulo, se hablará sobre los posibles formatos de exportación del lenguaje como son: Python con representación visual en consola, aplicaciones web en Django y sistemas de gestión como MySQL.

4.1. Clases abstractas y métodos para la generación de código

Para definir cómo se escribe el código de cada elemento del lenguaje, se definieron funciones genéricas para generar el código de la representación de los datos, la definición de consultas y la visualización de los datos. Estas funciones son **generate-data-code**, **generate-code** y **generate-visual-code**, y cada una recibe tres argumentos: un nodo, un formato de salida y un flujo de salida. El nodo representa el elemento del AST al que se le desea realizar la generación de código. El segundo argumento es una instancia de una clase que representa el formato de salida en el que se desea describir el código, y el tercer argumento es el flujo de salida al cual se desea escribir el código del nodo, en el formato de salida indicado.

Cada una de estas funciones mantiene la siguiente estructura:

```
(defgeneric generate-data-code (obj lang stream)
  (:documentation "Función genérica para generar el código de representación
                  de datos en el lenguaje dado."))
```

```
(defgeneric generate-code (obj lang stream)
  (:documentation "Función genérica para decidir cómo generar el código en
    el lenguaje dado."))

(defgeneric generate-visual-code (obj lang stream)
  (:documentation "Función genérica para generar el código de visualización
    en el lenguaje dado."))
```

Como se mencionó en la sección referente al lenguaje Lisp en el capítulo 1, los métodos proporcionan la implementación de las funciones genéricas; es por esto que para realizar el proceso de la generación de código es necesario implementar funciones para cada elemento del lenguaje como las que se muestran a continuación:

```
(defmethod generate-data-code ((obj node) (lang language) stream )
  ;; Instrucciones para generar el código
)

(defmethod generate-code ((obj node) (lang language) stream )
  ;; Instrucciones para generar el código
)

(defmethod generate-visual-code ((obj node) (lang language) stream )
  ;; Instrucciones para generar el código
)
```

Por ejemplo, a continuación se muestra cómo se puede implementar la generación de código del nodo de comparación de desigualdad a MySQL:

```
(defmethod generate-code ((obj clase-comparacion-desigualdad)
  (lang clase-export-sql)
  stream)
  (generate-code (izq obj) lang stream)
  (format stream " <> ")
  (generate-code (der obj) lang stream))
```

En el método anterior, primero se llama a **generate-code** sobre la propiedad *izq* del objeto para obtener el código del operando izquierdo. Luego, se escribe el operador de desigualdad *<>* y finalmente, se llama a **generate-code** sobre la propiedad *der* para obtener el código del operando derecho. Así, se construye la expresión completa de comparación en el formato adecuado.

De forma similar se puede generar el código referente a las entidades en Django, a continuación se presenta una posible implementación:

```
(defmethod generate-data-code ((obj clase-entidad)
                              (lang clase-export-django)
                              stream)
  (format stream "class ~a(models.Model):~%" (nombre obj))
  (format stream "~{    ~a = models.TextField()~%}~%" (atributos obj)))
```

En el ejemplo anterior, la función `generate-data-code` para entidades en Django define la clase correspondiente al modelo usando el nombre de la entidad (`nombre obj`) y recorre los atributos de la entidad (`atributos obj`) para declararlos como campos de texto en el modelo. Cada atributo se representa como una línea en la clase, utilizando `models.TextField()` como tipo de campo por defecto.

Un ejemplo de generación de código para la visualización de los datos en la consola de Python podría ser:

```
(defmethod generate-visual-code ((obj clase-pagina)
                                (lang clase-export-consola-python)
                                stream)
  (loop for elem in (elementos obj)
        do (generate-visual-code elem lang stream)))
```

En este caso, la función `generate-visual-code` recorre los elementos de la página y llama recursivamente a la misma función para cada elemento, generando así el código de visualización correspondiente a cada uno de ellos.

De esta manera, la estructura modular basada en funciones genéricas y métodos específicos permite adaptar la generación de código a distintos lenguajes y plataformas, facilitando la extensión y el mantenimiento del sistema. A continuación, se describen los principales formatos de exportación soportados por la herramienta y cómo cada uno responde a diferentes necesidades de gestión y visualización de las microaplicaciones.

4.2. Formatos para gestionar las microaplicaciones de gestión

Esta sección describe los principales formatos de exportación que soporta la herramienta, mostrando cómo la arquitectura de generación de código

digo permite adaptar la representación y gestión de las microaplicaciones a distintos entornos tecnológicos.

Se detallan los enfoques para consola de Python, aplicaciones web con Django y sistemas de gestión de bases de datos como MySQL, ilustrando cómo cada formato responde a diferentes necesidades de implementación y visualización.

4.2.1. Exportación a consola de Python

La exportación a consola de Python permite generar aplicaciones que pueden ejecutarse directamente en la terminal.

En este formato, las entidades se traducen a clases de Python y la información de cada entidad se almacena en un fichero de texto (.txt), donde cada línea representa una instancia de la entidad con sus atributos separados por comas.

Por ejemplo, la definición de una entidad en el lenguaje propuesto se realiza de la siguiente manera:

```
(def-entity estudiante (nombre grupo))
```

Esta definición se traduce automáticamente a una clase en Python que permite gestionar los datos de los estudiantes, cargar información desde un archivo y mostrarla en consola:

```
class estudiantes():
    def __init__(self, nombre, grupo):
        self.nombre = nombre
        self.grupo = grupo

    @staticmethod
    def cargar_datos(archivo):
        datos = []
        with open(archivo, 'r') as f:
            for line in f:
                nombre, grupo = line.strip().split(',')
                datos.append(estudiantes(nombre, grupo))
        return datos

    def __str__(self):
        return f'nombre: {self.nombre}, grupo: {self.grupo}'
```

Como se describe en el capítulo 3, la definición de consultas en el lenguaje propuesto, se basa en la composición de operadores lógicos, de comparación, iteraciones y accesos a atributos sobre entidades. Al exportar estas consultas al formato de consola de Python, cada consulta se traduce a una función de Python que recibe como argumentos los parámetros necesarios para filtrar los datos.

Por ejemplo, se tiene la siguiente consulta definida en el lenguaje como:

```
(def-consulta estudiantes_atendidos_por_profesor
  :argumentos (profesor_nombre)
  :iteraciones ((estudiante estudiantes)
                (asignacion asignaciones))
  :que-mostrar ((nombre-del-estudiante estudiante))
  :pregunta (se-cumple-simultaneamente
    (es-igual
      (profesor-de-la-asignacion asignacion)
      profesor_nombre)
    (es-igual
      (estudiante-de-la-asignacion asignacion)
      (nombre-del-estudiante estudiante))
    (es-igual
      (grupo-del-estudiante estudiante)
      "C412"))))
```

En este ejemplo, las iteraciones sobre las entidades se traducen a bucles anidados en Python. Los accesos a atributos se corresponden con los atributos de los objetos Python generados para cada entidad. Las condiciones lógicas y de comparación se implementan como expresiones booleanas y de comparación en las sentencias `if`. Finalmente, los resultados se agregan a una lista que se retorna al usuario.

El código que se genera a partir de la consulta anterior es el siguiente:

```
def estudiantes_atendidos_por_profesor(profesor_nombre):
    result = []
    for estudiante in estudiantes:
        for asignacion in asignaciones:
            if (asignacion.profesor == profesor_nombre)
                and (asignacion.estudiante == estudiante.nombre)
                and (estudiante.grupo == "C412"):
                result.append((estudiante.nombre))

    return result
```

La visualización definida en el lenguaje propuesto se genera a Python mediante funciones que imprimen los elementos visuales (títulos, textos, listas, ítems, etc.) utilizando instrucciones estándar de impresión como `print()`. Cada nodo visual del AST se convierte en una función o bloque de código Python que genera la salida correspondiente en la terminal.

Por ejemplo, una página definida en el lenguaje:

```
(pagina
  (titulo "MAG de Asignación de Alumnos Ayudantes")
  (texto "MAG para la asignación de estudiantes como ayudantes.")
  (lista
    (link-item "Consultar estudiantes atendidos por un profesor")
    (link-item "Consultar estudiante que imparte una asignatura"))
  (input "Seleccione una opción: "))
```

Tendría una salida en la consola de la siguiente manera:

```
=====
MAG de Asignación de Alumnos Ayudantes
=====
MAG para la asignación de estudiantes como ayudantes.

Opciones disponibles:
  1. Consultar estudiantes atendidos por un profesor
  2. Consultar estudiante que imparte una asignatura

Seleccione una opción: _
```

De igual forma, otros elementos visuales como listas, títulos o textos se representan mediante impresiones formateadas, y los campos de entrada (`input`) se traducen a llamadas a la función `input()` de Python. Así, la estructura visual definida en el lenguaje se mantiene en la consola, permitiendo una experiencia de usuario clara y ordenada, aunque adaptada a las limitaciones del entorno textual.

4.2.2. Exportación a aplicaciones web con Django

La exportación a Django permite transformar las microaplicaciones de gestión en aplicaciones web completas, aprovechando el framework Django para la gestión de modelos, vistas y controladores. En este caso, las entidades del lenguaje se traducen a modelos de Django, donde cada entidad

definida en el lenguaje propuesto (por ejemplo, (def-entidad estudiante (nombre grupo))) se convierte en una clase Python que hereda de `models.Model`. Los atributos de la entidad se representan como campos del modelo, utilizando los tipos de datos apropiados de Django (como `CharField`, `IntegerField`, etc.).

Por ejemplo, la entidad definida en el lenguaje:

```
(def-entidad estudiante nombre grupo)
```

se transpone a Django de la siguiente manera:

```
from django.db import models

class estudiantes(models.Model):
    nombre = models.CharField(max_length=100)
    grupo = models.CharField(max_length=100)
```

De igual forma, si en el lenguaje se define una relación entre entidades usando claves foráneas, como:

```
(def-entidad asignacion
  (profesor :foreign-key profesores nombre)
  (estudiante :foreign-key estudiantes nombre)
  (asignatura :foreign-key asignaturas nombre))
```

en Django se representa utilizando campos `ForeignKey` que enlazan con los modelos correspondientes:

```
class asignaciones(models.Model):
    profesor = models.ForeignKey(profesores, on_delete=models.CASCADE)
    estudiante = models.ForeignKey(estudiantes, on_delete=models.CASCADE)
    asignatura = models.ForeignKey(asignaturas, on_delete=models.CASCADE)
```

De esta manera, la estructura de entidades y relaciones definida en el lenguaje propuesto se refleja fielmente en los modelos de Django, permitiendo aprovechar el ORM para la gestión y consulta eficiente de los datos en la aplicación web.

En el lenguaje propuesto, la definición de consultas se realiza de manera declarativa utilizando la macro `def-consulta`. Al exportar la consulta a

Django, esta definición declarativa se transpone a una función o vista que utiliza el ORM de Django para realizar las operaciones de filtrado y consulta sobre los modelos correspondientes.

Por ejemplo, la siguiente consulta definida en el lenguaje como:

```
(def-consulta listar_asignaturas_impartidas_por_un_estudiante
  :argumentos (estudiante_arg)
  :iteraciones ((asignacion asignaciones))
  :que-mostrar ((asignatura-de-la-asignacion asignacion))
  :pregunta (es-igual
              (estudiante-de-la-asignacion asignacion)
              estudiante_arg))
```

genera código a Django de la siguiente manera:

```
def listar_asignaturas_impartidas_por_un_estudiante(request):
    estudiante_arg = request.GET.get('estudiante_arg')

    result = []
    asignaciones__ = asignaciones.objects.all()
    for asignacion in asignaciones__:
        if asignacion.estudiante == estudiante_arg:
            result.append((asignacion.asignatura))
    return render(request, 'page_name.html',
                  {'result': result, 'estudiante_arg': estudiante_arg})
```

Los resultados de la consulta se pasan a una plantilla HTML que se renderiza para mostrar los datos al usuario.

Por ejemplo, la visualización de los resultados de la consulta anterior en una plantilla HTML podría ser la siguiente:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Asignaturas impartidas por un estudiante</title>
</head>
<body>
  <h1>Asignaturas impartidas por el estudiante {{ estudiante_arg }}</h1>
  <ul>
    {% for asignatura in result %}
      <li> - {{ asignatura }}</li>
```

```
{% endfor %}  
</ul>  
</body>  
</html>
```

En este ejemplo, la variable `result` contiene la lista de asignaturas obtenidas por la consulta, y `estudiante_arg` es el nombre del estudiante recibido como argumento. La plantilla recorre la lista y muestra cada asignatura como un elemento de la lista HTML.

4.2.3. Exportación a sistemas de gestión de bases de datos como MySQL

La exportación a MySQL permite generar el esquema de base de datos necesario para almacenar la información gestionada por la microaplicación. En este formato, las entidades se traducen a sentencias `CREATE TABLE`, los atributos a columnas y las relaciones a claves foráneas según corresponda. Las consultas definidas en el lenguaje se transforman en sentencias `SELECT` que permiten recuperar la información deseada.

Por ejemplo, una entidad definida en el lenguaje se convierte en una tabla SQL:

```
CREATE TABLE estudiantes (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    nombre TEXT NOT NULL,  
    grupo TEXT NOT NULL  
);
```

Las consultas se traducen en instrucciones SQL que filtran y seleccionan los datos según las condiciones especificadas en el AST. Por ejemplo, la consulta presentada en la sección 4.2.1 podría generarse así:

```
DELIMITER //  
CREATE FUNCTION estudiantes_atendidos_por_profesor(  
    IN profesor_nombre VARCHAR(255))  
RETURNS TABLE  
BEGIN  
    RETURN  
    SELECT estudiantes.nombre  
    FROM estudiantes, asignaciones  
    WHERE (estudiantes.nombre = asignaciones.estudiante) AND
```

```
        (asignaciones.profesor = profesor_nombre) AND  
        (estudiantes.grupo = 'C412')  
    ;  
END //  
DELIMITER ;
```

Este formato facilita la integración con sistemas existentes y permite aprovechar la robustez y eficiencia de los motores de bases de datos relacionales.

La arquitectura basada en funciones genéricas y métodos específicos para cada formato permite que la herramienta sea fácilmente extensible. Si se desea soportar un nuevo formato de exportación, solo es necesario implementar los métodos correspondientes para ese formato, reutilizando la estructura del AST y la lógica de recorrido de nodos.

De esta manera, la herramienta se adapta a diferentes necesidades tecnológicas y permite que las microaplicaciones de gestión puedan ser desplegadas en diversos entornos, desde la consola hasta la web o sistemas de bases de datos, sin modificar la definición original de la aplicación.

Capítulo 5

Detalles de implementación

En este capítulo se presentan los detalles de implementación del lenguaje propuesto para la construcción de microaplicaciones de gestión (MAG). El objetivo es describir cómo se modelan y automatizan los distintos componentes del lenguaje, desde la definición de nodos básicos hasta la generación automática de estructuras complejas mediante macros.

5.1. Estructura general de los nodos del lenguaje

El lenguaje está basado en la construcción de un Árbol de Sintaxis Abstracta (AST), donde cada nodo representa una estructura específica, como textos, títulos, operadores lógicos, accesos a atributos, entre otros. La implementación de estos nodos se realiza a través de clases y funciones constructoras en Common Lisp, aprovechando la flexibilidad del lenguaje para definir diferentes tipos de argumentos.

5.1.1. Nodos con constructores de argumentos posicionales

Algunos nodos del lenguaje, como los que representan textos, títulos, entrada de datos, instancias de entidades, accesos a atributos o el operador lógico de negación, poseen constructores que reciben argumentos posicionales.

Por ejemplo, el nodo `titulo` recibe una cadena de texto con el contenido del título:

```
(titulo "Título de la página")
```

Otro ejemplo es el node not, que representa el operador lógico de negación y recibe una cláusula como argumento:

```
(no (es-igual (fecha-de-la-defensa defensa) "31 de Octubre"))
```

5.1.2. Nodos con constructores de argumentos &rest

Para otros nodos se emplean constructores que utilizan la opción &rest en su lista de argumentos, especialmente aquellos que pueden contener una cantidad variable de elementos como listas, elementos de listas, páginas u operadores lógicos de conjunción y disyunción.

Esto permite que los usuarios del lenguaje puedan pasar múltiples elementos de manera flexible, sin necesidad de agruparlos explícitamente en una lista.

Por ejemplo, el nodo pagina puede recibir una lista de elementos como argumentos:

```
(pagina  
  (titulo "Página de ejemplo")  
  (lista  
    (item "Elemento 1")  
    (item "Elemento 2")))
```

Otro ejemplo es el nodo and, que representa el operador lógico de conjunción y puede recibir múltiples cláusulas:

```
(al-mismo-tiempo  
  (es-igual (libro-de-la-nota nota) "Cien años de soledad")  
  (es-igual (etiqueta-de-la-nota nota) "Realismo mágico"))
```

En la siguiente sección se profundizará en la abstracción y generación automática de algunas de estas clases mediante las macros.

5.2. Automatización mediante macros

Para evitar la repetición de patrones en la definición de clases y constructores, se han desarrollado macros que automatizan la generación de código. Entre las más relevantes se encuentran `defclass*`, `def-entidad` y `def-consulta`.

5.2.1. Macro `defclass*`

La macro `defclass*` permite definir una clase y generar automáticamente un constructor por defecto. Esta macro simplifica la creación de clases al combinar la definición de la clase y su constructor en una sola expresión, reduciendo así la cantidad de código necesario.

Por ejemplo, al definir una clase llamada `ejemplo` con atributos, se puede utilizar la macro de la siguiente manera:

```
(defclass* ejemplo ()  
  (elementos)  
  :constructor-name ejemplo  
  :lambda-list (&rest elementos))
```

Esto generará automáticamente la clase `ejemplo` con un constructor llamado `new-ejemplo` que acepta una lista de elementos como argumentos. La macro también maneja la creación de los slots necesarios y sus accesores, facilitando el acceso a los atributos de la clase.

```
(defclass ejemplo ()  
  (elementos :accessor elementos :initarg :elementos))  
(defun new-ejemplo (&rest elementos)  
  (make-instance 'clase-ejemplo :elementos elementos))
```

Esta macro es especialmente útil para mantener la consistencia en la definición de clases y constructores, evitando la necesidad de escribir código repetitivo y propenso a errores.

5.2.2. Macro `def-entidad`

Si se desea acceder a los atributos de una entidad, debería definirse un acceso para cada uno de los atributos. Sin embargo, este enfoque presenta un problema significativo cuando se trabaja con múltiples entidades que poseen distintos atributos, ya que requeriría definir manualmente cada acceso a atributo de forma repetitiva, resultando en un proceso engorroso.

Para solucionar este problema, se diseñó la macro *def-entidad*, cuya función principal es automatizar la generación de los métodos de acceso a los atributos de las clases.

Un ejemplo sencillo de uso es el siguiente:

```
(def-entidad defensa (estudiante tutor oponente))
```

Esto generará automáticamente el siguiente código para la declaración de la entidad y los accesos a sus atributos:

```
(defparameter defensas
  (new-entidad
   :nombre "defensas"
   :atributos (list "estudiante" "tutor" "oponente")))

(defun estudiante-del-defensa (instancia-entidad)
  (acceso-a-atributo "estudiante" "defensas" instancia-entidad))

(defun tutor-del-defensa (instancia-entidad)
  (acceso-a-atributo "tutor" "defensas" instancia-entidad))

(defun oponente-del-defensa (instancia-entidad)
  (acceso-a-atributo "oponente" "defensas" instancia-entidad))
```

Para lograr una mejor experiencia de usuario, el lenguaje es capaz de establecer el acceso a atributo con un nombre gramaticalmente correcto. Por eso, además de las funciones anteriormente generadas para cada acceso a atributo, la macro también genera las siguientes funciones:

```
(defun estudiante-de-la-defensa (instancia-entidad)
  (acceso-a-atributo "estudiante" "defensas" instancia-entidad))
(defun tutor-de-la-defensa (instancia-entidad)
  (acceso-a-atributo "tutor" "defensas" instancia-entidad))
(defun oponente-de-la-defensa (instancia-entidad)
  (acceso-a-atributo "oponente" "defensas" instancia-entidad))
```

De esta manera, los accesos a los atributos de la entidad defensa mantienen una estructura más coherente de forma sintáctica.

Como convención del lenguaje, cabe destacar que se establece el nombre de la entidad en la declaración de la macro en singular. Si la entidad es Libros se pone libro o si es Defensas se pone defensa. El lenguaje es capaz de establecer el nombre de la entidad en plural por sí solo.

También el usuario tiene la posibilidad de establecer cómo quiere que se le llame a la entidad, agregando un segundo elemento en el primer argumento de la macro. Por ejemplo:

```
(def-entidad (defensa defensas) (estudiante tutor oponente))
```

O si se quiere establecer otro nombre diferente:

```
(def-entidad (defensa exposiciones) (estudiante tutor oponente))
```

El primer elemento defensa será utilizado para construir los accesos a los atributos; mientras que el segundo elemento será utilizado para crear las entidades defensas o exposiciones.

5.2.3. Macro `def-consulta`

Para definir una consulta se definió la clase *definicion-consulta* mediante la macro *defclass**. Esta clase posee como propiedades un nombre para la consulta, una lista de argumentos, una lista con las entidades que hay que iterar para obtener el resultado, una lista con los accesos a los atributos que se quieren mostrar y una pregunta o instrucción que identifica las condiciones de filtrado de la información. Esta última propiedad generalmente es un operador lógico o un operador de comparación.

Si se desea definir una consulta en el lenguaje, debería definirse cada instancia de entidad con antelación manualmente. Este proceso se torna repetitivo, por lo que se decidió redefinir la manera en que se define una consulta, es decir, que no fuera desde el constructor de la clase *definicion-consulta*.

Para solucionar este problema, se diseñó la macro *def-consulta*, cuya función principal es automatizar la creación de las instancias de las entidades.

Por ejemplo, si se tiene una consulta que sea obtener todos los profesores que tengan grado científico Dr, el usuario puede definir la consulta en el lenguaje propuesto de la siguiente forma:

```
(def-consulta profesores_con_grado_dr
  :argumentos ()
  :iteraciones ((profesor profesores))
  :que-mostrar ((nombre-del-profesor profesor))
  :pregunta (es-igual (grado-del-profesor profesor) "Dr"))
```

En resumen, la implementación del lenguaje propuesto se basa en la definición de nodos mediante clases y constructores flexibles, la automatización de patrones comunes mediante macros y la adopción de convenciones que facilitan la extensión y el mantenimiento del lenguaje. Estos detalles permiten construir microaplicaciones de gestión de manera eficiente, expresiva y adaptable a diferentes dominios.

Conclusiones

El trabajo realizado demuestra que es posible automatizar el desarrollo de microaplicaciones de gestión mediante un lenguaje de dominio específico, en este caso LDMAG, diseñado específicamente para ese propósito. Gracias a su implementación en Common Lisp, LDMAG aprovecha mecanismos potentes como las macros y el sistema de objetos CLOS para construir un lenguaje flexible, extensible y expresivo. Además, se comprobó su utilidad práctica mediante la generación automática de aplicaciones en distintos formatos, como consola, web (Django) y bases de datos SQL. Esto confirma la viabilidad del enfoque propuesto y su potencial para ahorrar tiempo y reducir la complejidad del desarrollo de sistemas de información en contextos académicos y organizacionales.

Recomendaciones

Como continuación de este trabajo, se recomienda ampliar las capacidades del lenguaje propuesto para que pueda generar aplicaciones en otros entornos, como bots de mensajería, con el fin de cubrir un espectro más amplio de necesidades. Asimismo, sería valioso desarrollar una herramienta con interfaz gráfica que permita diseñar aplicaciones de manera visual y genere automáticamente el código en LDMAg, facilitando aún más su uso por parte de usuarios no técnicos. Finalmente, se sugiere realizar una validación práctica del lenguaje en otros procesos académicos u organizacionales, lo cual permitiría detectar oportunidades de mejora y adaptar el lenguaje a contextos diversos.

Referencias Bibliográficas

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2006. (Citado en la página 10).
- [2] Alfred V. Aho and Jeffrey D. Ullman. *The Theory of Parsing, Translation, and Compiling*, volume Volume I of *Prentice-Hall series in automatic computation*. Prentice-Hall, 1972. Citado en la página 8. (Citado en la página 9).
- [3] E. J. Arreategui Zegarra, K. J. Fuentes Huaman, G. Herrera Landaveri, and F. C. Lillo Canales. Implementación de un sistema web para automatización del proceso de registro de notas del colegio apóstol santiago ventanilla, 2023, 2023. Trabajo académico. (Citado en la página 1).
- [4] D. C. Attaway. *Matlab: a practical introduction to programming and problem solving*. Butterworth-Heinemann, 2022. (Citado en la página 9).
- [5] Conrad Barski. *Land of Lisp*. No Starch Press, 2011. Citado en las páginas 9, 10 y 14. (Citado en la página 14).
- [6] O. S. Barzaga-Sablón, H. J. J. V. Pincay, J. V. Nevárez-Barberán, and M. V. A. Cobeña. Gestión de la información y toma de decisiones en organizaciones educativas. *Revista de Ciencias Sociales*, 25(2):120–130, 2019. (Citado en la página 1).
- [7] R. Batra. *An Introduction to SQL*, pages 1–9. Apress, Berkeley, CA, 2018. (Citado en la página 9).
- [8] E. J. Bolaños Pesantez and E. A. Vera Cuesta. Desarrollo de una aplicación web para gestión de matriculación y control de notas para el instituto nacional de tecnologías, 2024. (Citado en la página 1).

- [9] Dan Bornstein, Tom Nurkkala, Kyle Ferrio, and Guido van Rossum. *Lenguaje Implementation Patterns. Create Your Own Domain Specific and General Programming Languages*, volume 2010-1-4. The Pragmatic Bookshelf, 3 edition, 2009. Citado en las páginas 1, 7 y 8. (Citado en la página 8).
- [10] Django Software Foundation. *Django Documentation*, 2025. Accessed: June 15, 2025. (Citado en la página 2).
- [11] R. Elmasri, S. B. Navathe, V. C. Castillo, G. Z. Pérez, and B. G. Espiga. *Fundamentos de sistemas de bases de datos*. Pearson Educación, 2007. (Citado en la página 2).
- [12] Escuela Especializada en Ingeniería ITCA – FEPADE. *Informe Anual Programa de Investigación Aplicada, ITCA – FEPADE. Año 2009*. ITCA-EDITORES, Santa Tecla, El Salvador, 2012. (Citado en la página 11).
- [13] Á. F. Esquivel Valverde, R. León Robaina, and G. M. Castellanos Pallerols. Mejora continua de los procesos de gestión del conocimiento en instituciones de educación superior ecuatorianas. *Retos de la Dirección*, 11(2):56–72, 2017. (Citado en la página 1).
- [14] GeeksforGeeks. Parse tree and syntax tree. (Citado en la página 9).
- [15] José Luis Correa Godefoy. Confección automática de tribunales, June 2013. (Citado en las páginas 1, 7 y 18).
- [16] Paul Graham. *On Lisp*. Prentice Hall, 1994. (Citado en las páginas 11, 13 y 14).
- [17] T. P. Hart. Macro definitions for lisp. Ai memo, Massachusetts Institute of Technology, Artificial Intelligence Group, 1963. (Citado en la página 14).
- [18] Laravel. *Laravel Documentation*, 2025. Accessed: June 15, 2025. (Citado en la página 2).
- [19] Lisp-docs.github.io. Class precedence list. (Citado en la página 11).
- [20] Lisp-lang.org. Common lisp object system (clos) introduction. (Citado en la página 13).
- [21] Amalia López Rodríguez. Planificación automática de exámenes mundiales, Mayo 2013. Tesis de Diploma, Universidad de La Habana. (Citado en las páginas 2 y 7).

- [22] L. X. Martínez Rolán, A. Dafonte Gómez, and S. Garcia Miron. Usos de las aplicaciones móviles de mensajería en la docencia universitaria: Telegram. In *6° Congreso Internacional sobre Buenas Prácticas con TIC*, Málaga, España, October 2017. Comunicación audiovisual e publicidad. (Citado en la página 2).
- [23] M. Mernik. *Formal and Practical Aspects of Domain-Specific Languages: Recent Developments*. IGI Global, 2013. Citado en las páginas 1, 7 y 8. (Citado en la página 9).
- [24] M. Moreira Delgado. Gestión por procesos y su aplicación en la organización de información de empresa de telecomunicaciones de cuba. s.a. *Cienc Inf.*, 38(3), 2007. (Citado en la página 1).
- [25] A. Paepcke, editor. *Object-oriented programming: the CLOS perspective*, volume 9. MIT Press, Cambridge, 1993. (Citado en las páginas 4 y 11).
- [26] Pallets. *Flask Documentation*, 2025. Accessed: June 15, 2025. (Citado en la página 2).
- [27] Sergio Pérez Pantoja. Propuesta extensible para un sistema de gestión de un curso docente basado en créditos y entretenimiento, December 2023. (Citado en las páginas 2 y 7).
- [28] Raúl Portela Aguilera. Una aproximación a la generación automática de horarios para la facultad de matemática y computación de la universidad de la habana, Noviembre 2010. Tesis de Diploma, Universidad de La Habana. (Citado en las páginas 1 y 7).
- [29] Diamis Alfonso Pérez. Diseño e implementación de un sistema para la gestión de horarios docentes, January 2024. (Citado en las páginas 1 y 7).
- [30] M. H. Pérez and L. A. L. Céspedes. Bot de telegram para consultar información de la universidad de las ciencias informáticas. *Serie científica de la Universidad de las Ciencias Informáticas*, 14(7):82–95, 2021. (Citado en la página 3).
- [31] S. J. Ricapa Astorayme. Gerencia educativa y el monitoreo pedagógico de los docentes de la institución educativa n° 20356 jesús obrero-medio mundo, 2024. (Citado en la página 1).

- [32] Claudia Naveda Riesco, Alberto Cortez, Germán Montejano, and Daniel. Lenguaje específico de dominio para aplicaciones de negocios. Citado en la página 7. (Citado en la página 8).
- [33] D. Ruiz-Fuentes, R. M. Almaguer-Torres, I. C. Torres-Torres, and A. M. Hernández-Peña. La gestión por procesos, su surgimiento y aspectos teóricos. *Ciencias Holguín*, 2013. (Citado en la página 6).
- [34] Peter Seibel. *Practical Common Lisp*. Apress, 2005. (Citado en las páginas 11 y 12).
- [35] Guy Steele. *Common Lisp: The Language*. Elsevier, 1990. Citado en la página 20. (Citado en la página 11).
- [36] J. L. Suárez-Díaz, N. Rodríguez-Barroso, and G. Gómez-Trenado. Bot de telegram interactivo como herramienta de apoyo para la enseñanza. –, 2024. (Citado en la página 3).
- [37] Arnel Sánchez Rodríguez. Sistema para la gestión de un curso docente basado en créditos y entretenimiento, Noviembre 2022. Tesis de Diploma, Universidad de La Habana. (Citado en las páginas 2 y 7).
- [38] D. S. Touretzky. *Common Lisp: A Gentle Introduction to Symbolic Computation*, volume 1. The MIT Press, 1990. (Citado en la página 4).
- [39] Edmund Weitz. *Common Lisp Recipes: A Problem-Solution Approach*. Apress, 2016. (Citado en la página 4).
- [40] K. Wilson. *Introduction to HTML*, pages 31–39. Apress, Berkeley, CA, 2023. (Citado en la página 9).
- [41] Juan Carlos Casteleiro Wong. Sistema de gestión docente para la facultad de matemática y computación de la universidad de la habana, November 2022. (Citado en las páginas 1, 7 y 17).